JAVA SDE-2 INTERVIEW PREPARATION SERIES

JAVA ENGINEERING - BOOK 09 OF 09 - STUDY STEP 18G

Advanced Java G: Question Bank, Study Plan, and Reference

Interview Questions, Mock Loops, Solutions, Glossary, References, and Final Readiness

BY

Vinay Reddy Kalluri

Convert the core Java path into active recall, coding drills, mock interviews, evidence-based revision, and an SDE-2 readiness decision.

QUESTION BANK | MOCK LOOPS | SOLUTIONS | GLOSSARY | REFERENCES

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `JAVA 08 — Advanced Java E — Diagnostics`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Part G is the core Java revision system. It consolidates interview questions, selected solutions, study loops, terminology, and primary references without replacing deliberate practice.

Readiness map

Decision	Evidence
START HERE	Knowledge exists but explanations are not concise; A mock loop exposes repeated categories of mistakes; Revision needs prioritization; A claim needs a primary source.
PREPARE FIRST	Complete or selectively review Study Steps 01-17 and 18A-18F; Maintain an error log and evidence notebook.
MOVE ON WHEN	Answer SDE-2 Java questions with structured depth; Run coding, debugging, design, and behavioral mock loops; Use solutions as delayed feedback; Create a targeted final revision plan.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Java Segment Position

JAVA 08 - Advanced Java E - Diagnostics	**YOU ARE HERE** **JAVA 09 - Advanced Java G - Interview Revision**	SEGMENT COMPLETE

Contents

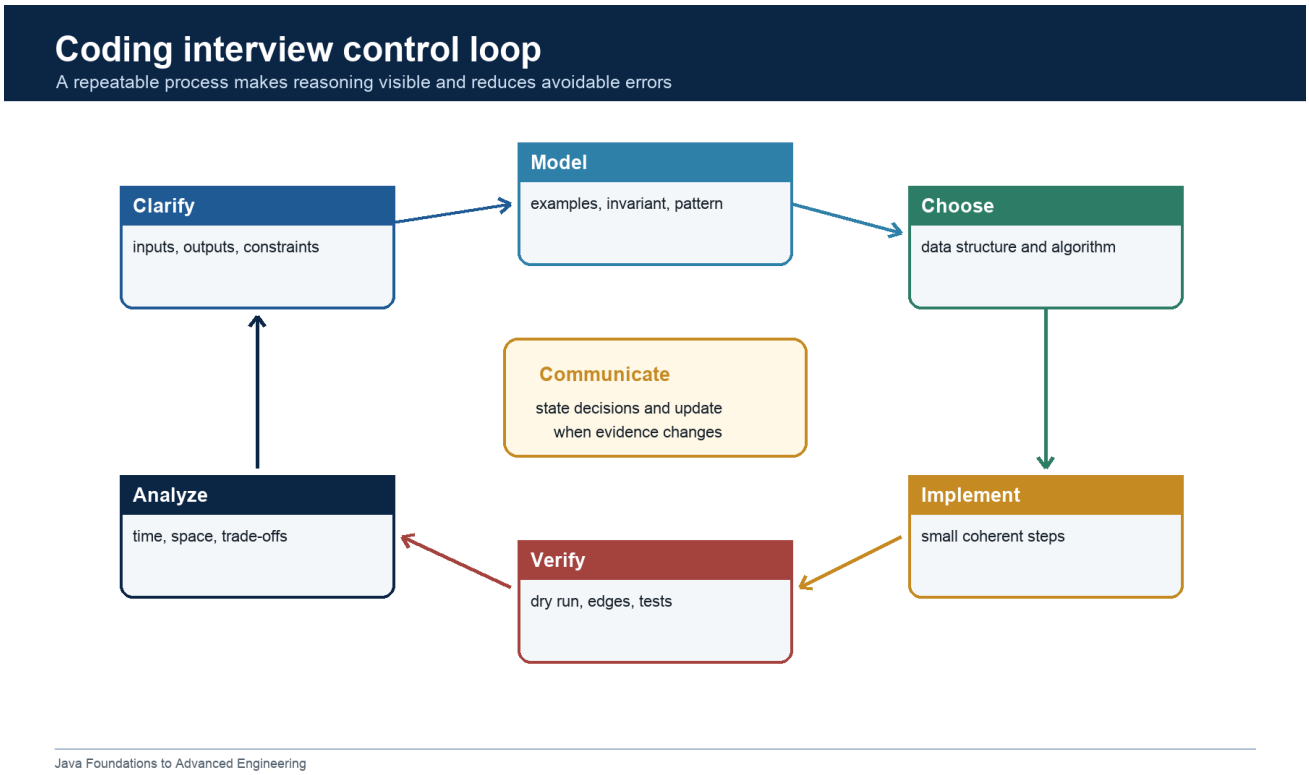
This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - The Java Coding Interview Playbook



A repeatable coding-interview control loop from clarification through analysis

Learning objectives

By the end of this chapter, you should be able to:

- run a repeatable control loop from clarification through final complexity;
- convert examples and constraints into a baseline, pattern, invariant, and proof;
- communicate while coding without narrating every keystroke;
- write compact, compilable Java 21 with deliberate numeric and collection choices;
- test with a boundary matrix and recover systematically from defects; and
- handle optimization, follow-ups, and production questions at SDE-2 depth.

WHY IT WORKS | Why this matters at SDE-2

Knowledge does not automatically become interview performance. Time pressure causes candidates to skip the contract, silently assume input properties, start coding a half-remembered pattern, and discover an off-by-one error near the end. A playbook preserves reasoning bandwidth.

The interviewer is evaluating several channels at once: correctness, problem decomposition, complexity, communication, code quality, testing, and response to feedback. A candidate who reaches code slightly later but keeps the solution controlled often performs better than one who types immediately and debugs by instinct.

This chapter is not a script to recite. It is a feedback loop. Each artifact - contract, example, baseline, invariant, dry run, and complexity statement - makes the next decision safer and gives the interviewer a chance to correct a misunderstanding early.

> **Focused practice path:** The 18-step companion series turns this playbook into shorter topic loops. Begin with the first book whose completion check is not yet automatic, and use Study Step 18G for question-bank, mock-loop, and final revision work.

WHY IT WORKS | First-principles model

A coding interview is a constrained engineering session. The problem statement defines an incomplete contract. Input bounds create a performance budget. The solution is a claim that an implementation satisfies the clarified contract within that budget. Evidence consists of an invariant or recurrence, a dry run, tests, and complexity analysis.

Treat the session as a control system:

TEXT

```
observe requirements -> propose model -> receive feedback -> implement
         ^                                          |
         +------------ trace and verify <-----------+
```

At every stage, maintain a usable fallback. The brute-force baseline is not wasted time: it validates the target behavior, reveals repeated work, and can become a test oracle. If optimization fails, a correct partial solution with honest analysis is better than an unverified sophisticated one.

> **Specification boundary:** Java guarantees defined language and library behavior, but not the judge's memory limit, stack size, default input constraints, or expected method signature. Those belong to the interview contract and must be clarified or stated as assumptions.

Core terminology

- **Clarifying question:** resolves an ambiguity that can change correctness or design.
- **Constraint pressure:** reason the baseline fails at stated input scale.
- **Pattern signal:** structural property suggesting a technique, not proof by keyword.
- **Invariant:** statement connecting variables to the processed and unprocessed state.
- **Oracle:** trusted implementation or property used to verify another result.
- **Boundary matrix:** compact set of tests covering shape, value, and behavioral extremes.
- **Counterexample:** input disproving a proposed rule or invariant.
- **Follow-up:** changed constraint requiring adaptation or trade-off discussion.
- **Recovery loop:** reproduce, localize invariant violation, repair, and retrace.
- **Closeout:** final proof, complexity, mutation, edge cases, and alternative summary.

Detailed mechanics

A 45-minute operating rhythm

Exact interview lengths vary, but a time budget prevents uncontrolled drift:

Phase	Approximate time	Deliverable
Clarify and examples	4-6 minutes	Contract and expected cases
Baseline and optimization	5-8 minutes	Costed alternatives and selected approach
Invariant and pseudocode	3-5 minutes	Proof skeleton
Java implementation	15-20 minutes	Compilable solution
Dry run and tests	5-7 minutes	Corrected boundary behavior
Follow-ups and close	Remaining	Complexity and trade-offs

Do not obey the table mechanically. A familiar problem can move faster; a modeling-heavy graph problem deserves more clarification. The important discipline is noticing when ten silent minutes have passed without a verified artifact.

Step 1: clarify the contract

Ask questions that can change the algorithm:

- Can input be null or empty?
- What are n and value ranges?
- Are values sorted, unique, positive, or immutable?
- Do duplicates represent separate items?
- May the method mutate or reorder input?
- Which output is required when several are valid?
- Is there always a solution? What represents absence?
- Are indexes, values, paths, counts, or only an optimum required?
- What does "character," interval overlap, or distance mean?

Avoid spending time on irrelevant possibilities. If the interviewer says to choose reasonable assumptions, state them aloud and continue: "I will treat input as non-null, allow an empty array, and return -1 when infeasible."

Restate the contract in one sentence. This creates an early synchronization point.

Step 2: construct examples

Use one normal example, one smallest case, and one adversarial case tied to a likely mistake. For a window, include a case that shrinks multiple times. For a BST, include a deep ancestor violation. For Dijkstra, include a stale priority-queue entry. For duplicates, include equal values on both sides of a boundary.

Calculate expected output manually before coding. If that is difficult, the contract is not yet understood. Do not rely exclusively on the sample supplied by the interviewer; samples often avoid exactly the boundary that breaks a memorized template.

Step 3: establish and pressure-test a baseline

Describe the direct correct approach in enough detail to cost it. For example: enumerate every subarray, calculate each sum incrementally, and track the best, O(n squared) time and O(1) space. Then compare with constraints. At n = 200, it may be fine. At n = 200,000, it is not.

Name eliminated work:

- repeated membership test -> hash set;
- repeated range aggregate -> prefix sum;
- repeated valid-range rescan -> sliding window;
- repeated minimum selection -> heap;
- repeated subproblem -> memoization/DP;
- exhaustive ordered boundary -> binary search;
- repeated connectivity traversal -> DSU.

This explanation is stronger than saying "I know this pattern." It shows why the representation pays for itself.

Step 4: select a pattern and state its preconditions

Use a decision map, then check its proof obligation:

Need	Candidate	Must be true
Complement/frequency/history	Hashing	Equality and memory acceptable
Eliminate ordered candidates	Two pointers	Pointer movement is monotone and safe
Maintain contiguous valid region	Window	State updates and validity permit shrinking
Ordered boundary/answer	Binary search	Predicate is monotone
Nearest in edge count	BFS	Edges have equal effective weight
Next greater/smaller	Monotonic stack	Popped candidate is permanently dominated
Best k/frontier minimum	Heap	Only priority head is required
Repeated optimal states	DP	State is sufficient; transitions complete

If a precondition fails, do not force the pattern. Negative numbers can invalidate a sum window. Negative edges invalidate Dijkstra. A non-monotone feasibility test invalidates binary search.

Step 5: articulate the invariant before code

An interview invariant should be operational, not philosophical:

- "At loop start, indexes below `low` are infeasible and indexes at or above `high` are feasible."
- "The map counts prefix sums strictly before the current prefix."
- "The deque contains unexpired indexes in increasing index and decreasing value order."
- "`dp[a]` is the minimum number of coins for exact amount a, or impossible."

Also state progress: right advances, candidate interval shrinks, one node moves from suffix to prefix, or every recursive call reduces remaining work.

Write short pseudocode when it helps validate phase order. For mutation-heavy logic, draw state. For recursion, write the return contract and base case before the method body.

Step 6: implement interview-grade Java

Prefer Java that makes the proof visible:

- semantic names such as `left`, `rightExclusive`, `indegree`, and `prefixFrequency`;
- half-open intervals where they simplify boundaries;
- `long` before potentially overflowing arithmetic;
- `ArrayDeque` for stacks and queues;
- `PriorityQueue` with `Integer.compare`, `Long.compare`, or comparator factories;
- records for small immutable state pairs;
- arrays for dense integer domains and maps for sparse domains;
- helper methods where they isolate a predicate or traversal contract.

Do not build a production framework during a 40-minute exercise. Avoid streams when a stateful loop is easier to trace. Avoid clever one-liners, raw types, mutable static fields, and subtraction comparators. A local nested record is often clearer than parallel arrays, but check what the interview environment supports.

Compilation is a separate correctness layer. Scan imports, generic types, return paths, method names, and static context. If no IDE is available, mentally compile one declaration at a time.

Step 7: dry run by invariant

Do not narrate only values. At each iteration, verify the invariant and boundary updates. Use a small table with the variables that prove correctness. For recursive code, draw the first few frames and show state restoration. For a heap or deque, show contents in logical order, not internal library array order.

Select a case that activates the difficult branch. Testing a sliding window that never shrinks or a graph without a cycle proves little.

Step 8: use a boundary matrix

Cover categories rather than random anecdotes:

Dimension	Cases
Size	empty, one, two, typical, maximum shape
Values	zero, negative, duplicates, min/max numeric
Position	answer at start, middle, end, absent
Structure	sorted/reversed, skewed tree, disconnected graph, cycle
Behavior	impossible, multiple valid answers, all qualify, none qualify

Trace two or three high-yield cases aloud. Mention additional cases you would automate. If time permits, compare optimized output against the baseline on small generated inputs; this property-based oracle catches many pointer and DP defects.

Step 9: recover from a bug

Do not patch randomly. Use this loop:

1. Freeze a minimal failing input.
2. Identify the first step where actual state violates the invariant.
3. Classify the cause: initialization, update order, boundary, stale state, overflow, or contract mismatch.
4. Make the smallest coherent correction.
5. Restart the trace from initialization.
6. Recheck complexity and neighboring boundary cases.

Say what you found. "I am expiring the deque after reading its front, so an old maximum leaks into the answer. I will expire before reporting." This demonstrates control rather than panic.

Step 10: close and handle follow-ups

End with:

- time in named dimensions and any expected/amortized qualifier;
- auxiliary, recursion-stack, and output space;
- whether input is mutated;
- the critical precondition and invariant;
- one alternative and when it is preferable; and
- production constraints if asked.

When a follow-up changes one constraint, identify which proof breaks before changing code. If input becomes streaming, random access disappears. If memory becomes O(1), hashing may no longer fit. If negatives are allowed, window monotonicity can fail. This approach prevents cargo-cult adaptation.

TRACE IT | Worked Java example

Problem: each positive workload takes `ceil(work / rate)` hours at integer processing rate. Workloads are handled sequentially, and each nonempty workload consumes at least one hour. Return the minimum rate that completes all work within `hourLimit`, or -1 when impossible.

JAVA (continued 1/2)

```
public final class MinimumProcessingRate {
    public static int minimumRate(int[] workloads, int hourLimit) {
        if (workloads == null) throw new IllegalArgumentException("null workloads");
        if (workloads.length == 0) return 0;
        if (hourLimit < workloads.length) return -1;

        int high = 0;
        for (int workload : workloads) {
            if (workload <= 0) {
                throw new IllegalArgumentException("workloads must be positive");
            }
            high = Math.max(high, workload);
        }

        int low = 1;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (finishesWithin(workloads, hourLimit, mid)) {
                high = mid;
            } else {
                low = mid + 1;
            }
```

JAVA (continued 2/2)

```
        }
        return low;
    }

    private static boolean finishesWithin(
            int[] workloads, int hourLimit, int rate) {
        long hours = 0;
        for (int workload : workloads) {
            hours += (workload + (long) rate - 1) / rate;
            if (hours > hourLimit) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(minimumRate(new int[] {3, 6, 7, 11}, 8)); // 4
        System.out.println(minimumRate(new int[] {5}, 1));           // 5
        System.out.println(minimumRate(new int[] {2, 3}, 1));        // -1
        System.out.println(minimumRate(new int[] {}, 0));            // 0
    }
}
```

The feasibility predicate is monotone: if rate r finishes in time, every larger rate also finishes in time. The answer is therefore the first true rate in `[1, maxWorkload]`.

TRACE IT | Execution or memory walkthrough

For workloads `[3,6,7,11]` and limit 8, initialize candidate range `[1,11]` with both endpoints represented by `low` and `high`. Rate 11 is feasible because each workload takes one hour.

Step	low	high	mid	Hours needed	Update
1	1	11	6	6	`high = 6`
2	1	6	3	10	`low = 4`
3	4	6	5	8	`high = 5`
4	4	5	4	8	`high = 4`
End	4	4	-	-	Return 4

Invariant: every rate below `low` is infeasible, and `high` is feasible; the first feasible rate remains in `[low, high]`. A feasible mid can still be larger than minimum, so retain it by setting `high = mid`. An infeasible mid is excluded with `low = mid + 1`. The interval strictly shrinks.

The ceiling formula promotes before addition to avoid `int` overflow. Early exit in the predicate preserves correctness because hours only increase. The empty-input behavior and impossible case are explicit contract choices rather than accidental loop results.

Complexity and performance

Let n be workload count and M the maximum workload. Feasibility is O(n) time and O(1) space. Binary search performs O(log M) predicate calls, so total time is O(n log M) and auxiliary space is O(1). The value-domain logarithm is distinct from O(log n).

A brute-force scan of rates is O(nM) and is a useful conceptual oracle for small M. Sorting does not help because every feasibility check needs the aggregate work across all workloads. Precomputing cannot remove dependence on rate without a more specialized constraint set.

Interview choice	Benefit	Cost or risk
Baseline first	Correctness anchor and oracle	Uses a few minutes
Helper predicate	Isolates monotonic proof	Extra method boundary
`long` aggregate	Prevents realistic overflow	Must promote before arithmetic
Early predicate exit	Less work on infeasible rates	Does not change worst-case bound
Half-open/closed convention	Prevents boundary drift	Must remain consistent

> **HotSpot note:** HotSpot may inline the feasibility helper and optimize the primitive loop. Such optimization changes constants, not the O(n log M) algorithm or the need for overflow-safe arithmetic.

WATCH OUT | Edge cases and common mistakes

- Coding before deciding null, empty, impossible, and mutation behavior.
- Asking many questions that cannot affect the algorithm while missing value ranges.
- Quoting a pattern without stating its preconditions.
- Giving an optimized idea without a correct baseline or proof.
- Using sample input as the only test.
- Mixing `int` and `long` so overflow occurs before widening.
- Writing ceiling division as floating point and introducing precision or conversion issues.
- Failing to establish that the binary-search predicate is monotone.
- Mixing first-true updates with any-match loop conditions.
- Narrating syntax instead of decisions and invariants.
- Going silent after finding a bug and applying unexplained patches.
- Claiming success without tracing the branch that mutates or shrinks state.
- Reporting O(log n) when the search dimension is a value M.
- Forgetting stack or output space.
- Overengineering interfaces and classes that do not help solve or verify the problem.
- Treating interviewer feedback as a command to patch rather than evidence to re-evaluate the model.

Production engineering notes

Interview code optimizes for a short, isolated demonstration. Production code needs input limits, observability, cancellation, API error types, test suites, documentation, and ownership. Do not paste a coding solution directly into a service boundary.

Numeric models deserve special attention. A workload might exceed `int`, processing might be parallel rather than sequential, or hour limits might use time zones and partial units. Clarify units and use domain types where appropriate. The worked predicate assumes deterministic integer work and no setup cost.

For large inputs, benchmark the actual representation. A map-heavy expected O(n) solution may exceed memory. A recursive DFS may overflow a worker stack. A returned list may be the dominant allocation. State these constraints in an SDE-2 follow-up without derailing the initial interview solution.

Interview practice should be measurable. Track failure categories: contract, recognition, proof, coding, boundary, complexity, and communication. Re-solving ten problems without classifying mistakes is less useful than repairing one repeated invariant failure.

TRY IT | Interview questions and model answers

Should I start coding immediately if I recognize the problem?

No. Spend a short, bounded period confirming the contract, constraints, and expected output. Then state the approach and invariant. Recognition accelerates reasoning but does not replace proof or boundary checks.

How much should I talk while coding?

Explain decisions, invariant-changing updates, and trade-offs. Do not narrate punctuation. Brief silence while implementing a stated plan is fine; resynchronize when entering a tricky branch or after changing the plan.

What if I cannot find the optimal solution?

Present a correct baseline, analyze its bottleneck, and explore one optimization at a time. Ask for a constraint hint if appropriate. A verified suboptimal solution plus clear progress is stronger than speculative code.

What should I do when the interviewer finds a counterexample?

Restate the failing case, find the first invariant violation, explain which assumption failed, repair the model, and retrace. Do not defend the old approach or patch only the observed output.

How do I know binary search on the answer is valid?

Define a totally ordered candidate domain and a feasibility predicate. Prove all candidates on one side are false and all on the other side true. Then search explicitly for the first true or last false boundary.

What distinguishes an SDE-2 closeout?

It includes named dimensions, qualified time, auxiliary/stack/output space, mutation, critical invariant, edge cases, and a credible alternative. It can also discuss scale, overflow, concurrency, or API contracts when prompted.

TRY IT | Exercises

1. Run the ten-step playbook on two-sum, recording no more than one sentence per step.
2. Solve a variable-window problem and design a test that forces several left-pointer moves in one iteration.
3. Use an O(n squared) oracle to property-test an O(n) prefix-hash solution on small random arrays.
4. Conduct a 45-minute graph mock and record time spent in each phase.
5. Deliberately inject an off-by-one error into lower bound, then practice the recovery loop aloud.
6. Adapt `minimumRate` when workloads may be zero and when work can be split across workers; identify which assumptions change.
7. Take one recent failed problem and classify the root cause as contract, model, invariant, implementation, or testing.
8. Practice three follow-ups: forbid extra space, make input streaming, and require all valid outputs.

RECAP | Chapter summary

A reliable coding interview is a controlled engineering loop. Clarify the contract, compute examples, establish a costed baseline, name eliminated work, verify pattern preconditions, state an invariant and progress measure, implement readable Java, trace a difficult branch, test a boundary matrix, and close with complete complexity. When a defect appears, localize the first invariant violation rather than patching randomly. Follow-ups change constraints; identify which proof breaks before adapting code.

TRY IT | Revision checklist

- [ ] I can restate null, empty, duplicate, mutation, and absence behavior.
- [ ] I create a normal, smallest, and adversarial example before coding.
- [ ] I state a correct baseline and the repeated work being eliminated.
- [ ] I verify pattern preconditions rather than matching keywords.
- [ ] I articulate an operational invariant and progress measure.
- [ ] I use semantic Java names, safe arithmetic, and appropriate collections.
- [ ] I trace the difficult branch and use a boundary matrix.
- [ ] I recover by finding the first invariant violation.
- [ ] I report time, auxiliary space, stack, output, and mutation.
- [ ] I identify which assumption a follow-up changes before editing code.

SDE-2 CORE CHAPTER

Chapter 2 - SDE-2 Java Interview Question Bank

Learning objectives

- Answer Java questions with definition, mechanism, guarantee, trade-off, and production relevance.
- Recognize follow-up probes that distinguish memorization from working knowledge.
- Detect misconception traps involving specifications, implementation details, complexity, and concurrency.
- Practice across JVM internals, language engineering, collections, concurrency, performance, DSA, backend design, testing, and security.
- Convert weak answers into short, precise SDE-2 model answers.

WHY IT WORKS | Why this matters at SDE-2

An SDE-2 interview rarely stops at a definition. "HashMap is O(1)" invites collision, resizing, equality, and mutability probes. "Volatile gives visibility" invites a lost-update example. "The object is on the heap" invites escape analysis and the distinction between Java semantics and HotSpot optimization.

The interviewer is evaluating reasoning under uncertainty. A strong candidate states the contract first, labels implementation-specific details, tests assumptions, and connects the answer to a production decision. A weak candidate recites internal trivia without knowing whether it is guaranteed.

This bank is organized for active recall. Cover the model answer, speak for 60 to 120 seconds, then answer the follow-up before reading. Mark a question green only if the explanation is correct, bounded, and usable without prompting.

WHY IT WORKS | First-principles model

Use a five-part answer shape:

```text
1. Definition: What is it?
2. Contract: What does Java guarantee?
3. Mechanism: How is it commonly implemented?
4. Trade-off: When does it help or hurt?
5. Evidence/example: How would I prove, test, or diagnose it?
```

Do not force all five parts into the opening sentence. Lead with the direct answer, then expand according to the probe. For comparison questions, establish dimensions before choosing: correctness, complexity, contention, ordering, memory, startup, tail latency, maintainability, and operational evidence.

For coding questions, use a parallel loop:

```text
clarify contract -> examples -> baseline -> invariant -> algorithm
-> complexity -> implement -> test edges -> discuss alternatives
```

> **Specification boundary:** Interview answers should distinguish JLS/JVMS and Java API guarantees from HotSpot, OpenJDK library, operating-system, and hardware behavior. An implementation detail is useful only when labeled with version and purpose.

Core terminology

- **Opening answer:** Direct 20-to-40-second response before deeper probes.
- **Follow-up probe:** Question testing mechanism, edge cases, alternatives, or production application.
- **Misconception trap:** Plausible but false simplification, such as "Java passes objects by reference."
- **Invariant:** Property that must remain true throughout an algorithm or state transition.
- **Linearization point:** Instant at which a concurrent operation appears to take effect.
- **Compatibility:** Source, binary, class-file, runtime, or behavioral compatibility, depending on context.
- **Complexity qualification:** Expected, amortized, worst-case, or implementation-sensitive cost.
- **Readiness evidence:** Timed performance, explanation quality, test coverage, and corrected-error history rather than confidence alone.

Detailed mechanics

The following bank includes an opening model answer, one likely follow-up, and a trap. The answers are intentionally concise; a mock interviewer should ask for examples and counterexamples.

JVM and execution

1. What is the difference among JDK, JRE, and JVM?

Model answer: The JVM loads and executes class files. A runtime combines a JVM with standard modules, native support, and resources. A JDK contains a runtime plus tools such as `javac`, `jar`, `javadoc`, and diagnostics. Modern deployments may use a custom `jlink` runtime instead of a separately packaged historical JRE.

Follow-up: Why can code compiled on JDK 21 fail on Java 17? Trap: Saying a JRE always exists as a separate vendor download.

2. Trace Java source to CPU execution.

Model answer: `javac` lexes, parses, resolves names, type-checks, and emits class files. At runtime, loaders define classes; the JVM verifies, prepares, resolves, and initializes them. An execution engine interprets, JIT-compiles, ahead-of-time executes, or combines strategies before native instructions run.

Follow-up: Where can `NoSuchMethodError` occur? Trap: Calling Java only interpreted.

3. What is bytecode?

Model answer: Bytecode is the JVM's platform-neutral instruction representation stored in class-file method structures. It uses an operand-stack model and symbolic constant-pool references. It is not CPU machine code, and one source construct need not map to one bytecode sequence.

Follow-up: Use `javap -c` to explain `invokevirtual`. Trap: Calling the class-file constant pool the same as the string pool.

4. What happens during class loading?

Model answer: Loading finds bytes and creates a loader-scoped definition. Linking verifies safety, prepares static storage with defaults, and resolves symbolic references, possibly lazily. Initialization later executes static initializers on first required active use under a synchronized once-only protocol.

Follow-up: Does `SomeType.class` initialize the class? Trap: Saying explicit static values are assigned during preparation.

5. Why can two classes with the same name fail a cast?

Model answer: Runtime class identity includes the binary name and defining class loader. Two loaders that independently define `com.example.Message` produce different types. Plugin systems normally place shared contracts in a parent loader and implementations in isolated children.

Follow-up: Describe a class-loader leak. Trap: Assuming byte-for-byte identical definitions are assignment compatible.

6. Explain JVM runtime data areas.

Model answer: The JVM model has a shared heap for objects, shared per-class/method structures and runtime constant pools, and per-thread PCs, JVM stacks, frames, and native execution support. HotSpot structures such as Metaspace, TLABs, and code cache implement or extend this model but are not JVMS region names.

Follow-up: Why can process memory exceed `-Xmx`? Trap: Putting every static-referenced object in Metaspace.

7. What does a JIT compiler optimize?

Model answer: An adaptive JIT uses profiles to compile hot paths, inline calls, specialize receiver types, eliminate checks, analyze escape, and optimize loops. Guarded assumptions remain correct through fallback and deoptimization. Compilation consumes CPU and code-cache memory.

Follow-up: What is on-stack replacement? Trap: Assuming a source method call always has a physical frame.

8. What is a safepoint?

Model answer: It is an implementation safe state used for VM operations needing coordinated managed state. Relevant threads reach polls or transitions, then the operation runs. Time to safepoint and operation duration are separate; stop-the-world does not automatically mean full GC.

Follow-up: Name a non-GC safepoint operation. Trap: Calling safepoints a JLS feature.

Memory and garbage collection

9. What happens when `new` executes?

Model answer: The class must be ready, storage and identity are obtained, fields receive mandatory defaults, and constructor invocation runs superclass construction, instance initializers, and the body. TLAB allocation, header words, field offsets, and compressed references are HotSpot details.

Follow-up: Can allocation be eliminated? Trap: Claiming Java guarantees stack allocation.

10. Shallow size versus retained size?

Model answer: Shallow size is the storage directly occupied by one object. Retained size is the storage that becomes unreachable if that object is removed, determined by GC-root paths and dominance. Referenced objects do not automatically belong to a parent's retained set.

Follow-up: Why can a small cache entry retain megabytes? Trap: Summing declared field widths as retained size.

11. How does GC identify garbage?

Model answer: A tracing collector starts from VM-known roots and follows strong and other applicable reachability edges. Objects with no relevant root path become reclamation candidates. Unreachable cycles are collectible, unlike naive reference counting.

Follow-up: Name common GC roots. Trap: Saying an object is garbage when a local is assigned null regardless of other paths.

12. Compare strong, soft, weak, and phantom references.

Model answer: Strong references retain normally. Soft references are discretionary memory-sensitive references and poor deterministic cache bounds. Weak references do not retain weakly reachable referents. Phantom references always return null and support post-mortem cleanup coordination through a `ReferenceQueue`.

Follow-up: Why drain a reference queue? Trap: Relying on finalization or phantom references to close resources promptly.

13. Can Java have memory leaks?

Model answer: Yes. GC removes unreachable objects, not reachable objects the application no longer needs. Unbounded caches, listeners, queues, thread locals, and class loaders can retain useless graphs. Compare allocation rate with post-collection live-set and dominator evidence.

Follow-up: Heap leak versus native leak? Trap: Calling every high allocation rate a leak.

14. Compare G1, ZGC, and Parallel GC.

Model answer: Parallel GC prioritizes throughput with parallel stop-the-world work. G1 uses regions, evacuation, concurrent marking, and pause goals for a balance. ZGC performs most work concurrently for very low pauses. Exact capabilities, including generations, depend on JDK version.

Follow-up: Which would you choose for a 99.9 latency SLO? Trap: Selecting from heap size alone without workload evidence.

Java language and API design

15. Is Java pass-by-reference?

Model answer: No. Every argument value is copied into a parameter. For objects, that copied value is a reference, so caller and callee can mutate the same object. Reassigning the parameter cannot reassign the caller's variable.

Follow-up: Demonstrate with an array. Trap: Saying "objects are pass-by-reference" as shorthand.

16. Overloading versus overriding?

Model answer: Overloading selects among methods at compile time using declared types and conversions. Overriding supplies runtime instance dispatch based on the receiver's actual class. Static methods are hidden, not overridden; constructors are neither inherited nor overridden.

Follow-up: Which overload receives `null`? Trap: Treating return type alone as an overload distinction.

17. Interface versus abstract class?

Model answer: An interface defines a multiple-inheritance contract and can have default/static/private behavior but no per-instance constructor state. An abstract class shares state, protected implementation, and construction under single class inheritance. Choose by semantic relationship and evolution needs, not method count.

Follow-up: How do default-method conflicts resolve? Trap: Saying interfaces contain no implementation.

18. Composition versus inheritance?

Model answer: Inheritance creates a substitutability commitment and exposes superclass evolution. Composition delegates to a collaborator and keeps behavior replaceable. Prefer composition unless the subtype preserves the parent's behavioral contract and shared protected implementation is genuinely valuable.

Follow-up: Give a Liskov-substitution violation. Trap: Using inheritance only for code reuse.

19. What makes a class immutable?

Model answer: Its observable state cannot change after construction. Use private final fields, validate construction, prevent `this` escape, make defensive copies of mutable inputs/outputs, and ensure referenced components are immutable or encapsulated. A final reference alone is insufficient.

Follow-up: Are records deeply immutable? Trap: Equating `final` or record with deep immutability.

20. Explain `equals` and `hashCode`.

Model answer: Equal objects must produce the same hash code; unequal objects may collide. Equality should be reflexive, symmetric, transitive, consistent, and false for null. Keys must not change equality-relevant state while in hash collections.

Follow-up: Why can subclass equality break symmetry? Trap: Using database-generated IDs inconsistently across entity lifecycle.

21. Checked versus unchecked exceptions?

Model answer: Checked exceptions must be caught or declared and can express recoverable alternatives callers are expected to handle. Unchecked exceptions often represent violated contracts or failures not locally recoverable. The choice is API design, not severity; preserve causes and avoid broad catch-and-ignore.

Follow-up: When should an exception be translated? Trap: Logging and rethrowing at every layer.

22. What does try-with-resources guarantee?

Model answer: It closes initialized resources in reverse order on normal or abrupt completion. If body and close both throw, the body exception remains primary and close failures are suppressed. Resource ownership must still be clear.

Follow-up: How do you inspect suppressed exceptions? Trap: Assuming close exceptions replace the primary failure.

23. Explain generic type erasure.

Model answer: Java generics are mainly enforced at compile time; type variables are translated to bounds/Object with casts and bridge methods where needed. Parameterized types generally do not have distinct runtime classes, so `new T()` and `instanceof List<String>` are unavailable.

Follow-up: What is heap pollution? Trap: Saying all generic information disappears from metadata and reflection.

24. Explain PECS.

Model answer: For a parameterized structure that produces `T`, use `? extends T`; for one that consumes `T`, use `? super T`. It guides flexible APIs but does not mean a structure cannot both read and write in all designs.

Follow-up: Why can you add null to `List<? extends Number>` but not an Integer? Trap: Treating `List<Integer>` as a subtype of `List<Number>`.

25. What Java 17 and 21 features matter most?

Model answer: Java 17 provides a modern LTS baseline with records, sealed types, and finalized pattern matching for `instanceof`. Java 21 adds virtual threads, record patterns, switch pattern matching, and sequenced collections. Preview features must be labeled separately.

Follow-up: When not to use a record? Trap: Listing preview structured concurrency as final Java 21.

Collections, streams, and I/O

26. ArrayList versus LinkedList?

Model answer: ArrayList gives O(1) indexed access, amortized O(1) append, compact storage, and cache locality. LinkedList gives O(1) insertion only when an existing node position is already represented by an iterator, but lookup is O(n) and each node costs memory. ArrayList is the usual default.

Follow-up: Removing from the front? Trap: Choosing LinkedList for arbitrary middle insertion while ignoring traversal.

27. How does HashMap work?

Model answer: It spreads a key hash to choose a table bin, then uses `equals` within collisions. Expected lookup is O(1) under good distribution; resize is O(n), and collision behavior can use trees in current implementations. Capacity and load factor trade memory against collision/resizing.

Follow-up: Why are mutable keys dangerous? Trap: Claiming `hashCode` must be unique.

28. TreeMap versus HashMap?

Model answer: TreeMap maintains key order and navigation with O(log n) operations. HashMap offers expected O(1) lookup without ordering. TreeMap equality for keys is effectively based on comparator/compare-to zero, so ordering consistency with `equals` matters.

Follow-up: How do range queries work? Trap: Using subtraction in a comparator and risking overflow.

29. What does fail-fast mean?

Model answer: Many ordinary collection iterators detect some unsupported structural modifications and may throw `ConcurrentModificationException` on a best-effort basis. It is a bug detector, not a synchronization guarantee. Concurrent collections define weak or snapshot iteration instead.

Follow-up: Is iteration safe if no exception occurred? Trap: Catching the exception and retrying as concurrency control.

30. Comparable versus Comparator?

Model answer: Comparable defines a type's natural order. Comparator is an external strategy, allowing multiple orderings and composition. Both must satisfy ordering contracts; sorted sets/maps treat comparison zero as equivalent for membership/key uniqueness.

Follow-up: Handle nulls? Trap: Comparator inconsistent/transitivity violations.

31. Stream versus collection?

Model answer: A collection stores elements; a stream is a single-use computation pipeline over a source. Intermediate operations are lazy, terminal operations trigger traversal, and side-effect-free stateless operations compose safely. Parallel streams need splittable work, sufficient size, associative reduction, and execution-context care.

Follow-up: `map` versus `flatMap`? Trap: Reusing a consumed stream.

32. Why must reduction be associative?

Model answer: Parallel partitioning can group operands differently. An associative accumulator/combiner produces the same logical result under regrouping. Floating-point addition is not mathematically associative due to rounding, so exact reproducibility may require sequential or specialized algorithms.

Follow-up: Identity requirements? Trap: Mutating one non-thread-safe container in `forEach` on a parallel stream.

33. Optional best practices?

Model answer: Optional is primarily a return type for possibly absent values. Use `map`, `flatMap`, `orElseGet`, and explicit absence semantics. Avoid Optional fields/parameters in ordinary domain models unless a framework/contract justifies them; never use Optional to hide errors.

Follow-up: `orElse` versus `orElseGet`? Trap: Calling `get()` without proving presence.

34. Buffered I/O versus NIO?

Model answer: Buffering amortizes system calls for stream/channel I/O. NIO.2 adds paths, file operations, channels, buffers, selectors, and asynchronous facilities. A `ByteBuffer` has capacity, position, and limit; `flip` changes from writing into the buffer to reading from it.

Follow-up: Direct buffer trade-offs? Trap: Assuming one `read` or `write` transfers the entire requested amount.

Concurrency

35. What is happens-before?

Model answer: It is the JMM relation ordering memory effects. It comes from program order, volatile write/read, monitor unlock/lock, thread start/join, class initialization, and concurrent-library contracts plus transitivity. Wall-clock ordering or sleep is not enough.

Follow-up: Prove safe publication with volatile. Trap: Explaining only CPU caches.

36. What does synchronized guarantee?

Model answer: Mutual exclusion for the same monitor and memory ordering: unlock happens-before a later lock. It is reentrant and releases on block exit, including exceptions. All conflicting accesses must follow the same guard or another valid protocol.

Follow-up: `wait` versus `sleep`? Trap: Synchronizing readers and writers on different objects.

37. What does volatile guarantee?

Model answer: Volatile access is atomic and globally ordered for that variable; a write happens-before subsequent reads, publishing prior actions. It does not make read-modify-write or multi-field invariants atomic. It fits flags and immutable snapshot references.

Follow-up: Why is `volatile count++` unsafe? Trap: Calling volatile a lightweight universal lock.

38. How does interruption work?

Model answer: Interruption is cooperative cancellation through status. Interruptible waits often throw `InterruptedException` and clear status. Propagate it, or restore status and exit at a boundary. It cannot safely force arbitrary code or every I/O call to stop.

Follow-up: Why restore status? Trap: Catch, log, and continue.

39. ReentrantLock versus synchronized?

Model answer: Both provide exclusion and visibility. ReentrantLock adds timed/interruptible acquisition, multiple conditions, and optional fairness but requires `finally` unlock. Use synchronized for simple scoped locking; choose explicit features from a requirement, not presumed speed.

Follow-up: Why await in a loop? Trap: Unlocking outside finally.

40. Explain CAS and ABA.

Model answer: CAS installs an update only if current state matches expected; a successful CAS is a linearization point. Retry loops must recompute without side effects. ABA occurs when state changes A-B-A and equality hides history; use a version/stamp or different design.

Follow-up: Is lock-free starvation-free? Trap: Assuming GC eliminates logical ABA.

41. How do you size an executor?

Model answer: Start near cores for CPU-bound work. For blocking work, estimate blocking fraction but cap by downstream capacity, memory, and latency. Use bounded queues, explicit rejection/backpressure, deadlines, metrics, and load tests. An unbounded queue can make maximum threads irrelevant.

Follow-up: Explain same-pool starvation deadlock. Trap: Maximizing threads without considering database connections.

42. `thenApply` versus `thenCompose`?

Model answer: `thenApply` maps `T` to `U`; `thenCompose` maps `T` to a completion stage and flattens it. Non-Async callbacks may run on the completing/attaching thread. Explicit executors isolate blocking and capacity. Recovery stages can accidentally hide failures.

Follow-up: `handle` versus `whenComplete`? Trap: Assuming CompletableFuture cancellation interrupts work.

43. Why virtual threads?

Model answer: Java 21 virtual threads make thread-per-task blocking code scalable by unmounting blocked tasks from carrier platform threads. They do not add CPU cores or downstream capacity. Do not pool them; limit scarce resources directly. In Java 21, blocking while pinned in synchronized/native code can occupy carriers.

Follow-up: Migration incident with JDBC? Trap: Removing all concurrency limits.

44. ConcurrentHashMap guarantees?

Model answer: It supports thread-safe concurrent operations and atomic methods such as `putIfAbsent`, `compute`, and `merge`. Iteration is weakly consistent, not a whole-map snapshot. Multi-key invariants still require another protocol. Null keys/values are not allowed.

Follow-up: Why is `containsKey` then `put` racy? Trap: Using `size` for a precise admission decision during mutation.

Performance, DSA, design, backend, testing, and security

45. How do you benchmark Java code?

Model answer: Define the decision and metric, use representative data, separate warm-up and measurement, consume results, fork JVMs, control environment, and report distributions. Use JMH for microbenchmarks because it addresses dead-code elimination, constant folding, and harness effects.

Follow-up: Why can `nanoTime` around one loop lie? Trap: Inferring production impact from a nanobenchmark.

46. CPU is high. What do you inspect?

Model answer: Confirm scope and timeline, then separate application, GC, JIT, native, and system CPU. Capture JFR or repeated profiles, correlate hot stacks with traffic and changes, and inspect runnable threads. Thread dumps alone sample stacks but not proportional CPU reliably.

Follow-up: What if CPU is low but latency high? Trap: Tuning GC without evidence.

47. What is the SDE-2 DSA method?

Model answer: Clarify input/output and constraints, run examples, state brute force, identify the invariant/data structure, derive complexity, implement incrementally, and test boundaries. Explain why the algorithm works, not only its pattern name.

Follow-up: How do you recover when stuck? Trap: Coding before clarifying duplicates, overflow, or mutation rules.

48. BFS versus DFS?

Model answer: BFS explores by distance layers and gives shortest path in unweighted graphs, using O(width) queue space. DFS explores depth, supports cycle/topological/component reasoning, and uses O(depth) stack/explicit storage. Both are O(V+E) with adjacency lists.

Follow-up: Why mark visited on enqueue? Trap: Recursive DFS on adversarial depth without stack analysis.

49. What makes a good backend API?

Model answer: Clear contract, validation, stable error model, idempotency where retries occur, pagination/bounds, authorization, observability, and compatibility. Keep transport DTOs separate from mutable persistence concerns and define timeout/cancellation behavior across dependencies.

Follow-up: Design idempotent payment creation. Trap: Treating HTTP retry as harmless without a key/state machine.

50. Explain transaction isolation and Java locks.

Model answer: A Java lock coordinates threads sharing one lock instance. A database transaction coordinates persisted operations across clients under an isolation level. In-process synchronization does not prevent another service instance from changing the row; use database constraints, locking/version checks, and idempotency.

Follow-up: Optimistic locking failure handling? Trap: Holding a Java lock across a remote transaction and calling it distributed consistency.

51. How would you test concurrent code?

Model answer: Prove invariants and happens-before/linearization points, then coordinate actor starts with barriers/latches, use bounded timeouts, record histories, and stress across forks/architectures. jcstress-style outcome tests can expose JMM behaviors. Passing tests do not prove absence of races.

Follow-up: How detect deadlock? Trap: Adding sleep to force order.

52. Unit versus integration versus contract tests?

Model answer: Unit tests isolate deterministic logic and run fast. Integration tests validate boundaries such as database, serialization, and framework wiring. Contract tests verify provider/consumer assumptions across service evolution. Use the cheapest layer that can catch the failure, with a smaller number of realistic end-to-end tests.

Follow-up: Test transaction rollback? Trap: Mocking the boundary whose semantics are under test.

53. What security checks belong in a Java service?

Model answer: Authenticate identity, authorize the specific resource/action, validate bounded input, parameterize SQL, avoid unsafe deserialization, protect secrets, constrain outbound requests, patch dependencies/JDK, and log security events without sensitive data. Apply least privilege and fail closed.

Follow-up: Prevent SSRF in a URL-fetch endpoint. Trap: Treating validation as only regex syntax.

54. How do you prevent injection?

Model answer: Keep data separate from interpreter syntax: prepared SQL parameters, contextual output encoding, safe command APIs without shell concatenation, and allowlisted structure when identifiers cannot be parameterized. Validation helps policy but escaping alone is context-sensitive and fragile.

Follow-up: Can prepared statements parameterize table names? Trap: Building shell commands from quoted user strings.

TRACE IT | Worked Java example

An interviewer asks: "Implement a small bounded LRU cache and discuss concurrency." A focused baseline is:

JAVA

```java
import java.util.LinkedHashMap;
import java.util.Map;

public final class LruCache<K, V> {
    private final int capacity;
    private final LinkedHashMap<K, V> entries;

    public LruCache(int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException("capacity");
        this.capacity = capacity;
        this.entries = new LinkedHashMap<>(16, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
                return size() > LruCache.this.capacity;
            }
        };
    }

    public synchronized V get(K key) {
        return entries.get(key);
    }

    public synchronized void put(K key, V value) {
        entries.put(key, value);
    }

    public synchronized int size() {
        return entries.size();
    }
}
```

`accessOrder=true` means a successful `get` mutates order, so even reads require synchronization. The intrinsic lock gives a simple linearization point per method. This is an in-process cache, not a distributed cache, and it deliberately omits loader coalescing, expiration, metrics, null policy, weight-based capacity, and external callback safety.

TRACE IT | Execution or memory walkthrough

For capacity two:

1. `put(A,1)` inserts A. Order is `[A]`.
2. `put(B,2)` inserts B. Order is `[A,B]`, least to most recently used.
3. `get(A)` returns 1 and moves A: `[B,A]`.
4. `put(C,3)` temporarily produces `[B,A,C]`.
5. `removeEldestEntry` observes size three and removes B, leaving `[A,C]`.

If T1 executes `get(A)` while T2 executes `put(C)`, the same monitor serializes the full LinkedHashMap operations. Without synchronization, a `get` can race because access-order maintenance changes linked structure. Returning mutable values still lets callers race on those values; cache thread safety does not make value objects immutable.

An excellent interview discussion adds that eviction callbacks must run after releasing the lock, loading should avoid duplicate work without holding the lock across I/O, and production caches need tested libraries unless the exercise explicitly requires implementation.

Complexity and performance

Expected `get` and `put` are O(1); eviction is O(1) for one eldest entry. Space is O(capacity), excluding retained key/value graphs. The single lock caps parallel operations and can create contention, but it provides a short proof. Segmenting/sharding can increase concurrency at the cost of approximate global LRU.

For the question bank, track more than answer count. Score each response from 0 to 4:

- 0: no viable answer.
- 1: memorized fragment or materially incorrect.
- 2: correct opening but weak mechanism/edge cases.
- 3: correct, structured, one follow-up handled.
- 4: precise contract, trade-off, production example, and follow-ups.

Require at least 3 on correctness-critical areas: JMM, equality/hashing, exceptions/resources, complexity, transactions, and security. Fast wrong answers are worse than slower bounded reasoning.

WATCH OUT | Edge cases and common mistakes

- Starting with five minutes of context instead of answering the question.
- Inventing a guarantee when uncertain; state the stable contract and label the uncertainty.
- Giving Big-O without expected/amortized/worst-case qualification.
- Describing HotSpot object headers or HashMap thresholds as language guarantees.
- Using a framework slogan instead of Java mechanics.
- Optimizing before defining the invariant and constraints.
- Ignoring null, empty, duplicate, overflow, cancellation, and adversarial inputs.
- Saying "thread-safe" without naming the operation/invariant.
- Presenting an incident with no measurement, decision, or outcome.
- Arguing that a data race is safe because it worked on x86.
- Hiding a weak area behind excessive terminology.

Production engineering notes

Prepare six reusable incident stories: latency, correctness, memory/GC, concurrency, dependency failure, and security/reliability. Each should state scale, signal, hypothesis, evidence, action, trade-off, and measured result. Never disclose employer secrets; normalize identifiers and approximate scale where needed.

For JVM questions, mention the tool that would verify the claim: `javap` for class files, JFR/profile for CPU/allocation/locks, GC logs and heap analysis for memory, thread dumps for blocking, and build/runtime metadata for compatibility. Tool names without an investigation sequence are not enough.

> **HotSpot note:** Questions about TLABs, mark words, C1/C2, safepoint polls, collector defaults, and virtual-thread diagnostics are version-specific. A strong answer says "in mainstream HotSpot on the stated JDK" before explaining them.

TRY IT | Interview questions and model answers

Use this twelve-question rapid loop after completing the bank:

1. **Explain source to CPU in 90 seconds.** Model: compiler, class file, loading/linking/init, adaptive execution, native code, specification boundary.
2. **Diagnose process OOM with stable heap.** Model: inspect container/RSS, stacks, Metaspace, direct buffers, code cache, native libraries, and NMT where available.
3. **Design an immutable key.** Model: stable equality/hash, final state, defensive copies, no mutable identity fields.
4. **Choose ArrayList or LinkedList.** Model: operation distribution and locality; ArrayList default.
5. **Prove volatile publication.** Model: program order, volatile synchronizes-with, transitive happens-before.
6. **Fix a shutdown hang.** Model: stop admission, interrupt/close blocking resource, propagate cancellation, join/await deadline, capture survivors.
7. **Bound an executor.** Model: downstream capacity, queue, rejection/backpressure, deadlines, metrics.
8. **Explain virtual-thread migration risk.** Model: cheap waiting increases concurrency; preserve DB/API bulkheads and diagnose Java 21 pinning.
9. **Solve longest unique substring.** Model: sliding window with last-seen indices, O(n) time and O(character-set) space.
10. **Design idempotent create.** Model: client key, atomic uniqueness, stored outcome/state machine, request-hash conflict policy.
11. **Investigate p99 regression.** Model: compare timeline/distributions, profile, queues/locks/GC/downstream, controlled experiment.
12. **Prevent unsafe deserialization.** Model: simple schema-bound formats, type allowlists, validation, dependency patches, least privilege, size/depth limits.

TRY IT | Exercises

1. Record answers to all 54 questions; score them 0 to 4 and keep the first incorrect sentence for review.
2. For ten questions, produce a 30-second, 90-second, and five-minute answer.
3. Add two follow-ups and one misconception trap to every question scored below 3.
4. Implement and test the LRU cache, then redesign loader coalescing without holding a lock during I/O.
5. Run three paired mocks: JVM/concurrency, collections/DSA, and backend/performance/security.
6. Build a one-page error log containing the guarantee you confused, a counterexample, and the corrected rule.
7. Select five implementation-specific answers and attach an exact JDK/VM version label.

RECAP | Chapter summary

SDE-2 answers begin with a direct contract, then explain mechanism, trade-offs, and evidence. The bank spans the execution pipeline, memory, language, collections, concurrency, performance, DSA, backend design, testing, and security because interviews connect these domains. Active recall, follow-up probes, misconception traps, and scored corrections convert knowledge into interview performance.

TRY IT | Revision checklist

- [ ] I can answer every bank question in under two minutes before follow-ups.
- [ ] I distinguish Java guarantees from HotSpot and library implementation details.
- [ ] I qualify complexity as expected, amortized, or worst-case.
- [ ] I can prove concurrency answers with happens-before or a linearization point.
- [ ] I connect JVM/performance answers to diagnostic evidence.
- [ ] I have production stories with measured outcomes and clear ownership.
- [ ] I handle coding constraints, edges, complexity, implementation, and tests aloud.
- [ ] No correctness-critical bank category scores below 3.

SDE-2 CORE CHAPTER

Chapter 3 - Eight-Week Study Plan and Mock Interview Loops

Learning objectives

- Convert the book into an eight-week schedule with daily outputs and weekly readiness gates.
- Balance Java depth, DSA execution, backend design, diagnostics, and behavioral stories.
- Run realistic mock loops and score them consistently.
- Use spaced retrieval and an error log instead of passive rereading.
- Recover from missed days, weak mocks, plateaus, and burnout without abandoning the plan.

WHY IT WORKS | Why this matters at SDE-2

Interview preparation fails when study feels productive but never tests retrieval under time. Reading concurrency twice is not the same as proving a happens-before edge aloud. Solving a problem after seeing its pattern is not the same as clarifying, deriving, coding, and testing in 40 minutes. Completing 200 random problems can coexist with weak Java depth and poor communication.

This plan treats preparation as an engineering system. Each week has inputs, outputs, measurements, and a gate. Mocks are feedback, not final exams. The schedule assumes an experienced backend engineer with work obligations and about 15 focused hours per week. It can be compressed or extended without changing the order of learning and verification.

WHY IT WORKS | First-principles model

Use a closed feedback loop:

TEXT

```
learn a model
    -> retrieve without notes
    -> implement or diagnose
    -> explain under a time limit
    -> receive evidence-based feedback
    -> record the smallest corrected rule
    -> retrieve again after spacing
```

Every study block must create an artifact: solved code, a spoken answer, a diagram from memory, a scored mock, an incident timeline, or a corrected error card. Highlighting pages is input, not evidence of mastery.

The plan has four parallel tracks:

Track	Outcome
Java/JVM	Precise contracts, internals, trade-offs, and diagnostics
Coding/DSA	Correct medium problems in 35-45 minutes with explanation/tests
Design/backend	APIs, data, concurrency, reliability, and scaling decisions
Communication/behavioral	Structured answers, ownership stories, and collaborative reasoning

One track must not consume every hour. An SDE-2 loop often mixes coding, Java/concurrency, design, and behavioral evidence.

Core terminology

- **Active recall:** Producing an answer without looking at notes.
- **Spaced retrieval:** Recalling material after increasing delays, such as 1, 3, 7, and 14 days.
- **Interleaving:** Mixing related problem types so pattern selection must be inferred.
- **Error log:** Short record of an incorrect decision, root cause, corrected rule, and retest date.
- **Mock loop:** Timed simulation followed by scoring and a correction cycle.
- **Readiness gate:** Objective threshold required before increasing interview intensity.
- **Red flag:** Correctness-critical weakness that cannot be offset by a high average.
- **Maintenance set:** Small recurring review set preventing mastered topics from decaying.
- **Recovery plan:** Predefined response to schedule or performance disruption.

Detailed mechanics

Before Day 1: establish the baseline

Reserve 90 minutes and do not study first:

1. Answer 20 mixed questions from Master Chapter 53, two minutes each.
2. Solve one unseen medium array/hash problem in 40 minutes.
3. Explain one production incident in five minutes.
4. Draw source-to-CPU and JVM memory from memory.
5. Score with the rubrics later in this chapter.

Create an error log with columns: date, domain, prompt, observed error, root cause, corrected rule, smallest counterexample, retest dates, and status. Keep answers short. "Review concurrency" is not actionable; "I claimed volatile increment is atomic; retest with a two-thread lost-update trace on D+1/D+3/D+7" is.

Choose one Java 21 JDK distribution and record its exact version. Build a small scratch project with tests, compiler warnings, and commands for `javap`, JFR, and thread dumps. Interviews may use an online editor, so practice both IDE and plain-editor execution.

> **Specification boundary:** Treat finalized Java 21 language/API behavior separately from preview features, vendor tools, and interview-platform restrictions. Preview APIs require explicit enablement and can change; a study schedule or hiring rubric is an editorial policy, not a Java platform guarantee.

Standard daily schedule

The working-engineer schedule is about 15 hours per week:

Monday through Friday, 2 hours:

- 15 minutes: spaced recall cards and one diagram.
- 35 minutes: focused chapter study.
- 45 minutes: implementation, diagnosis, or one coding problem.
- 15 minutes: speak one model answer without notes.
- 10 minutes: tests, error log, and next retrieval dates.

Saturday, 3 hours: 60-minute mock, 30-minute break/debrief, 75-minute targeted repair, 15-minute plan update.

Sunday, 2 hours: weekly retrieval test, story rehearsal, maintenance problems, and deliberate rest. Stop early if the gate is met and fatigue is high.

For a 90-minute weekday, keep 10 minutes recall, 30 minutes concept, 35 minutes code, and 15 minutes explanation/log. Do not remove retrieval or explanation; reduce scope. For a full-time search, add a second independent block after a long break, not a five-hour continuous session.

Week 1: computing model and JVM foundations

Goal: explain how Java executes and where memory/diagnostic evidence belongs.

Day	Primary work	Required output
1	Baseline assessment and environment	Scores, error log, exact JDK/runtime record
2	Why Java, JDK/JVM, compatibility, tools	90-second source-to-runtime answer
3	Compilation, bytecode, class files	Annotated `javap -c` for one class
4	JVM architecture and runtime data areas	Memory diagram from memory
5	Class loading and initialization	Failure table: CNFE, NCDFE, linkage/init errors
6	Object layout, calls, recursion	Coding mock plus reference/frame dry run
7	GC and JIT overview	Collector comparison and weekly retrieval score

Implementation tasks: compile with `--release 17`, inspect bytecode, create a static-initialization failure safely, capture a thread dump, and use a small JFR recording. Do not tune flags. The purpose is evidence literacy.

Gate 1: score at least 3/4 on source-to-CPU, class lifecycle, runtime areas, object creation, and GC reachability. Solve one medium array/hash problem in 50 minutes with correct complexity. If the JVM diagram still mixes Metaspace with heap guarantees, repeat the diagram on Days 8 and 10.

Week 2: Java language engineering

Goal: write and explain precise Java 17/21 code, equality, exceptions, generics, and object design.

Day	Primary work	Required output
8	Primitives, numeric semantics, operators	Overflow/promotion prediction test
9	Methods, overloading, varargs, pass-by-value	Five call-resolution dry runs
10	Arrays, strings, Unicode, text blocks	String/array memory and complexity explanation
11	Classes, access, inheritance, composition	Refactor inheritance to composition
12	Interfaces, sealed types, records, equality	Immutable value type with tests
13	Exceptions, resources, nested types, reflection	Suppressed-exception and reflection exercise
14	Generics, erasure, lambdas, Java 21 features	Language deep-dive mock and error repair

Complete two DSA problems this week involving strings/two pointers. For every solution, test empty input, one element, duplicates, boundary indices, and overflow where applicable. Explain when a record is inappropriate, why a final collection is still mutable, and one heap-pollution example.

Gate 2: average 3/4 across language mock dimensions with no red flags on pass-by-value, equality/hash contract, exception resource handling, or generic variance. Implement a correct immutable class in 25 minutes. If overload/generic reasoning is weak, use ten small compile-prediction snippets rather than rereading entire chapters.

Week 3: collections, streams, and I/O

Goal: select a data structure from operations and explain implementation-sensitive costs.

Day	Primary work	Required output
15	Collection hierarchy and List trade-offs	Selection table for six workloads
16	HashMap/HashSet internals	Hash/equality dry run and mutable-key failure
17	TreeMap/TreeSet and ordering	Three valid comparators with tests
18	Queue, deque, priority queue, heaps	Top-K implementation and complexity
19	Sorting, selection, Comparable/Comparator	Comparator contract review
20	Streams, collectors, Optional	Sequential pipeline plus safe collector
21	NIO.2, buffers, serialization boundaries	File-copy/buffer state exercise and mock

Coding maintenance: one sliding-window problem, one heap/top-K problem, and one interval/sorting problem. Repeat one older problem from memory after seven days; do not count immediate re-solves as mastery.

Gate 3: choose ArrayList/HashMap/TreeMap/heap/deque correctly from a new scenario, state qualified complexity, and solve two collection-heavy medium problems within 45 minutes each on separate days. Explain why fail-fast is not thread safety and why parallel streams are not a universal speed switch.

Week 4: concurrency and the Java Memory Model

Goal: prove visibility/atomicity and design cancellation, locking, scheduling, and virtual-thread capacity.

Day	Primary work	Required output
22	Threads, states, interruption, shutdown	Owned-worker shutdown protocol
23	Happens-before, volatile, safe publication	HB graph for two publication patterns
24	Intrinsic locks, wait/notify	Predicate-loop bounded buffer
25	ReentrantLock/Condition, deadlock	Lock-order transfer and wait-for graph
26	Atomics, CAS, ABA, accumulators	CAS dry run and linearization point
27	Executors, bounded queues, futures	Executor capacity plan and shutdown code
28	Concurrent collections, CompletableFuture, virtual threads	Full concurrency mock and incident diagnosis

Run one stress exercise, but pair it with a proof: a lost-update counter, safe volatile publication, or condition wait. Capture thread dumps from a deliberate safe deadlock and a healthy idle executor; learn the difference. On Java 21, create virtual threads and demonstrate a semaphore protecting a simulated downstream.

Gate 4: no score below 3 on interruption, happens-before, lock/condition loops, executor admission, and virtual-thread capacity. You must explain why `volatile count++` fails, why `Future.get(timeout)` does not cancel, and why a virtual-thread migration can overload JDBC. This is a hard gate; concurrency misconceptions should delay interviews involving deep Java.

Week 5: DSA foundations in Java

Goal: make problem-solving communication and core linear structures automatic.

Day	Primary work	Required output
29	Complexity and interview method	Verbal template plus brute-force comparison
30	Arrays, hashing, prefix sums	Two mediums, one timed
31	Two pointers and sliding windows	Invariant written before implementation
32	Linked lists	Reverse, cycle, and merge dry runs
33	Stacks, queues, monotonic structures	Next-greater/interval exercise
34	Trees and recursive reasoning	Traversal plus depth-risk discussion
35	BSTs and heaps	60-minute coding mock and repair

Problem selection should be representative, not random. For each pattern, solve one learning problem with notes, one related problem without notes after a day, and one mixed problem where the pattern is not named. Maintain Java hygiene: use `ArrayDeque` for stack/queue, avoid subtraction comparators, handle integer overflow, and choose meaningful helper methods.

Gate 5: in three unseen mediums, achieve two correct solutions within 45 minutes and one viable solution with minor correction. Every solution must include examples, invariant, complexity, and tests. If recognition is good but implementation fails, reduce new problems and do line-by-line dry runs plus typed reimplementation from a blank file.

Week 6: advanced DSA and low-level design

Goal: handle graph/state-space problems and translate requirements into maintainable Java objects.

Day	Primary work	Required output
36	Graph representation, BFS/DFS	Component/shortest-path comparison
37	Topological sort and union-find	Dependency ordering implementation
38	Weighted shortest paths	Dijkstra assumptions and stale-entry trace
39	Backtracking and pruning	State-choice-undo invariant
40	Greedy reasoning and counterexamples	Exchange argument or rejected greedy
41	Dynamic programming	State/transition/base/order derivation
42	SOLID, API design, patterns	60-minute LLD mock plus coding maintenance

Use one graph, one backtracking, and two DP problems; quality beats volume. For LLD, practice a rate limiter, parking lot, task scheduler, or notification service. State scope, concurrency, extension points, failure behavior, and what belongs outside the process.

Gate 6: solve one unseen graph medium and one DP medium with correct state/complexity; reach 3/4 on an LLD rubric for requirements, model, APIs, invariants, extensibility, and testability. A memorized pattern without a correctness explanation does not pass.

Week 7: backend, performance, reliability, and full loops

Goal: connect Java mechanics to service design and production evidence.

Day	Primary work	Required output
43	JDBC, connection pools, transactions	Transaction/isolation failure walkthrough
44	Serialization, APIs, idempotency	Idempotent create state machine
45	Performance method and JMH	Valid benchmark plan, no premature tuning
46	jcmd, JFR, dumps, GC incidents	Investigation playbook for p99/OOM/high CPU
47	Testing, build tools, dependencies	Test pyramid and reproducible build record
48	Security and reliability	Threat review for one API
49	Full coding + Java + design loop	Scores, recording, top-three repair plan

Prepare a story bank: difficult bug, performance improvement, conflict/trade-off, ownership beyond role, failed decision and learning, ambiguous project, and mentoring/collaboration. Each story should be five minutes with a 30-second summary. Include metrics but avoid confidential detail.

Gate 7: complete a full loop without a correctness red flag. Diagnose one incident using a timeline and tools, design idempotency/transactions correctly, and score at least 3 on testing/security. If system design is broad but shallow, practice one subsystem deeply rather than adding more architecture boxes.

Week 8: simulation, consolidation, and interview taper

Goal: produce stable performance, not learn an entirely new curriculum.

Day	Primary work	Required output
50	Full mock loop 1	Independent scores and error priorities
51	Repair top correctness weakness	Counterexample, implementation, retest
52	Full mock loop 2	Compare score trend and fatigue
53	Repair communication/timing	30/90-second answer set
54	Company-shaped mixed loop	Coding plus likely Java/design emphasis
55	Light maintenance and logistics	Environment, questions, story cards, sleep plan
56	Retrieval only and rest	No heavy new problems; final readiness decision

Schedule real interviews only after a gate if possible, with lower-priority companies before top choices but without treating any interviewer disrespectfully. Leave recovery time between loops. The final 48 hours should reduce volume, preserve sleep, and rehearse opening frameworks, not attempt a new advanced topic.

Mock loop formats

Coding mock, 60 minutes: 5 minutes clarification/examples, 5 minutes baseline/invariant, 35 minutes implementation, 10 minutes tests/complexity, 5 minutes feedback. The interviewer should not rescue pattern recognition immediately.

Java deep dive, 45 minutes: six rapid questions, two deep follow-up trees, one code/concurrency dry run, and five-minute summary feedback.

Low-level design, 60 minutes: requirements/scope 10, model/API 15, core flows/invariants 15, concurrency/failure/scaling 10, trade-offs/testing 10.

Backend/system design, 60 minutes: requirements/SLOs 10, estimates/interfaces 10, data/workflows 15, scaling/reliability 15, deep dive/trade-offs 10.

Behavioral, 45 minutes: four stories with follow-ups on ownership, conflict, failure, data, and learning. Feedback must identify missing context, personal action, trade-off, and outcome.

Debrief within 30 minutes: score first without notes, compare interviewer score, identify the earliest wrong decision, write one corrected rule, and schedule a retest. Do not immediately redo the same prompt from memory and call it fixed; use a transfer problem first.

TRACE IT | Worked Java example

This small Java 21-compatible tracker encodes a readiness gate instead of relying on mood:

JAVA (continued 1/2)

```java
import java.util.List;

public final class ReadinessTracker {
    public record MockScore(
            int correctness,
            int communication,
            int complexity,
            int testing,
            int javaDepth,
            int design) {

        public MockScore {
            int[] values = {
                    correctness, communication, complexity,
                    testing, javaDepth, design
            };
            for (int value : values) {
                if (value < 0 || value > 4) {
                    throw new IllegalArgumentException("score must be 0..4");
                }
            }
        }

        boolean clearsCriticalGate() {
```

JAVA (continued 2/2)

```java
            return correctness >= 3
                    && communication >= 3
                    && javaDepth >= 3
                    && testing >= 2;
        }
    }

    static double weighted(MockScore score) {
        return score.correctness() * 0.25
                + score.communication() * 0.15
                + score.complexity() * 0.15
                + score.testing() * 0.10
                + score.javaDepth() * 0.20
                + score.design() * 0.15;
    }

    static boolean ready(List<MockScore> scores) {
        if (scores.size() < 3) return false;
        List<MockScore> recent = scores.subList(scores.size() - 3, scores.size());
        return recent.stream().allMatch(MockScore::clearsCriticalGate)
                && recent.stream().mapToDouble(ReadinessTracker::weighted)
                        .average().orElse(0.0) >= 3.2;
    }
}
```

Weights are a planning policy, not a universal hiring rubric. Adjust them for the target role, but never let a high design score average away incorrect/racy code. Keep red-flag gates.

TRACE IT | Execution or memory walkthrough

Suppose the last three mock scores are:

TEXT

```
M1 = correctness 3, communication 3, complexity 3,
     testing 2, javaDepth 3, design 3       weighted 2.90

M2 = correctness 4, communication 3, complexity 3,
     testing 3, javaDepth 3, design 3       weighted 3.25

M3 = correctness 4, communication 4, complexity 3,
     testing 3, javaDepth 4, design 3       weighted 3.60
```

All three clear the critical minima, but the average is 3.25, so `ready` returns true. If M3 had Java depth 2, the average might remain acceptable, but `clearsCriticalGate` would fail and readiness would be false.

The method examines only the latest three mocks so old baseline failures do not dominate forever. That also means mock selection must be representative. Three easy array problems cannot establish system-design or concurrency readiness. Store domain-specific scorecards alongside the aggregate.

At runtime, record instances and list elements are ordinary objects; no concurrency is implied. If multiple evaluators update a shared tracker, publish immutable score-list snapshots or use an appropriate thread-safe store. The example calculates a decision; it does not persist evidence.

Complexity and performance

`weighted` is O(1). `ready` examines three recent records, effectively O(1), with O(1) auxiliary space aside from stream machinery. A generalized window of `k` mocks would be O(k).

The standard schedule invests about 15 hours/week, or 120 hours over eight weeks. Protect those hours from context switching. Two uninterrupted 60-minute blocks usually produce more evidence than four scattered half-hours. Track completed outputs and scores, not chair time.

Spaced retrieval should be selective. A new error is recalled after approximately 1, 3, 7, and 14 days; mastered cards move to weekly maintenance. If review consumes more than 25 percent of study time, retire duplicates and keep rules with counterexamples rather than accumulating trivia.

Problem count is a weak throughput metric. A better weekly dashboard includes unseen timed correctness, median time to viable approach, implementation defect count, Java question score, mock trend, sleep/fatigue, and number of unresolved red flags.

WATCH OUT | Edge cases and common mistakes

- Reading the entire book sequentially without retrieval or implementation.
- Solving only familiar tagged problems and overestimating pattern recognition.
- Counting a solution read as a problem solved.
- Doing mocks without written scores or repeating mocks without repairing root causes.
- Cramming missed work into one exhausted weekend.
- Letting DSA consume all Java, design, or behavioral time.
- Changing resources every few days instead of completing one feedback loop.
- Memorizing HotSpot internals without contracts or production evidence.
- Scheduling top-choice interviews before concurrency/correctness red flags clear.
- Treating a weighted average as permission to ignore a score of 1 in security or correctness.
- Sacrificing sleep; fatigue directly harms working memory, communication, and coding accuracy.
- Studying company trivia instead of role requirements and reusable fundamentals.

Production engineering notes

Treat preparation artifacts like an engineering repository: dated scorecards, runnable code, failing/passing tests, diagrams, and a changelog of corrected rules. Keep personal/employer data out of examples. Use synthetic incidents when a real story is confidential.

Before each interview, verify JDK/editor assumptions, meeting link, time zone, backup network/audio, allowed references, and whether coding occurs in a shared editor. Prepare concise questions about team ownership, reliability expectations, deployment, on-call, technical debt, mentorship, and success in the first six months.

> **HotSpot note:** Practice implementation-specific diagnostics on the same Java 21 runtime you recorded, but rehearse answers at the portable contract level first. Tool output and defaults can differ in the interviewer's JDK.

Recovery plans

Missed one or two days: Do not double the next day. Preserve recall, the week's mock, and the gate-critical topic. Drop optional new problems and move one deep dive to Sunday.

Lost a full week: Extend the calendar by a week if possible. If the interview date cannot move, retain Weeks 1-4 correctness foundations, core DSA, one design loop, and two final mocks; reduce breadth, not verification.

Repeated coding failure: Classify: misunderstood contract, wrong pattern, broken invariant, Java implementation defect, or time management. Do two untimed derivations, one blank-file implementation, then one transfer problem timed. Avoid ten more random problems.

Weak Java deep dive: Build 30/90-second answers and draw mechanisms. For each wrong claim, attach a counterexample and specification/implementation label. Retest verbally with follow-ups.

Mock anxiety: Increase exposure gradually: record alone, peer with known questions, peer with unseen questions, then full loop. Keep the same opening framework so the first two minutes are automatic.

Plateau: Inspect the error distribution. If the same root cause repeats, change the drill. If errors are diverse, reduce cognitive load and improve rest. Seek external feedback because self-scoring may be miscalibrated.

Burnout or sleep loss: Take a full rest day, cut volume by one third for a week, preserve only recall and one mock, and restore sleep/exercise. Extending the plan is cheaper than reinforcing careless habits.

TRY IT | Interview questions and model answers

How do I know I am ready?

Use three representative recent mocks with weighted average at least 3.2/4, no critical score below the gate, two unseen coding mediums solved correctly under time on different days, and one successful Java/concurrency plus design loop. Confidence is supporting information, not the gate.

How many coding problems should I solve?

Enough to demonstrate transfer across core patterns. Fifty deeply reviewed mixed problems can outperform 200 copied solutions. Track unseen timed correctness, explanation, tests, and delayed re-solves rather than a universal count.

Should I postpone an interview after a failed mock?

Look at failure type and timing. One difficult prompt is noise; a repeated correctness red flag in concurrency, coding, or design is signal. If rescheduling is feasible, repair and pass a transfer mock first. Do not wait for perfection.

What if I have only four weeks?

Run the baseline, combine Weeks 1-2, Weeks 3-4, Weeks 5-6, and Weeks 7-8. Keep one mock and debrief each week. Reduce problem breadth and optional internals, but retain JMM, collections, core DSA, backend correctness, and final simulations.

How should mocks be scored?

Use observable behavior: correct assumptions, invariant, implementation, tests, complexity, communication, depth, trade-offs, and recovery from hints. Score independently before discussion and record the earliest wrong decision, not only final outcome.

What should I do the day before?

Light retrieval, one easy confidence problem, logistics, story headlines, questions for interviewers, normal food/exercise, and sufficient sleep. No long mock, new advanced topic, or late-night cramming.

TRY IT | Exercises

1. Run the baseline and create the first eight error cards with D+1/D+3/D+7 dates.
2. Customize the 56-day table to your interview format without deleting any critical gate.
3. Build scorecards for coding, Java deep dive, LLD, backend design, and behavioral rounds.
4. Implement `ReadinessTracker`, add tests for insufficient mocks, boundary scores, and one red flag.
5. Schedule the first four mocks now, including reviewer and debrief time.
6. Write recovery versions for 90 minutes/day, one missed week, and a fixed interview in 14 days.
7. Record one 90-second JVM answer and one five-minute incident story; score and repeat after three days.
8. At the end of each week, publish a private one-page review: evidence, red flags, repaired rules, next gate.

RECAP | Chapter summary

An effective eight-week plan alternates learning with retrieval, implementation, explanation, mocks, and correction. Each week produces artifacts and ends at an objective gate. Java/JVM foundations and concurrency correctness precede high interview volume; DSA, design, backend reliability, testing, security, and behavioral evidence remain parallel tracks. Scored mock trends and critical minima determine readiness, while predefined recovery plans keep one disruption from destroying the schedule.

TRY IT | Revision checklist

- [ ] I completed a no-study baseline and created an actionable error log.
- [ ] My calendar contains daily outputs, weekly mocks, debriefs, and rest.
- [ ] I use 1/3/7/14-day retrieval for corrected rules.
- [ ] Every week has a measurable gate and recovery action.
- [ ] My coding practice includes unseen timed transfer problems.
- [ ] I score Java, coding, design, testing/security, and communication separately.
- [ ] I have at least three representative recent mocks with no critical red flag.
- [ ] My production stories include evidence, trade-offs, actions, and measured outcomes.
- [ ] I have a reduced-volume plan for missed time or burnout.
- [ ] The final 48 hours prioritize retrieval, logistics, and sleep.

SDE-2 CORE CHAPTER

Chapter 4 - Exercise Hints and Selected Solutions

E.1 How to use this appendix

These are not scripts to memorize. Each solution demonstrates an interview method:

1. restate the contract and assumptions;
2. identify the invariant;
3. choose a representation;
4. derive correctness before complexity;
5. name edge cases and failure behavior; and
6. test with a small dry run.

For each exercise, stop after the hint and attempt a solution. Compare reasoning before comparing syntax. A different solution is valid when it satisfies the stated contract and you can defend its costs. Code targets Java 17 unless a Java 21 feature is explicitly identified.

The self-check criteria are intentionally strict. An interview answer is not complete merely because the sample returns the expected output. It must state why the result generalizes and what the code promises at production boundaries.

E.2 JVM exercise: class initialization order

Problem. Predict the output. Explain which events are guaranteed by the Java language and which physical loading or compilation details remain implementation-specific.

JAVA

```java
public final class InitializationOrderExercise {
    static class Parent {
        static int parentValue = mark("Parent field");

        static {
            System.out.println("Parent block");
        }

        static int mark(String text) {
            System.out.println(text);
            return 9;
        }
    }

    static class Child extends Parent {
        static final int CONSTANT = 7;
        static Integer boxed = mark("Child field");

        static {
            System.out.println("Child block");
        }
    }

    public static void main(String[] args) {
        System.out.println(Child.CONSTANT);
        System.out.println(Child.boxed);
    }
}
```

Hint. Separate loading/linking from initialization. Then ask whether each field read is an active use and whether the field is a compile-time constant variable.

Selected solution. The expected output is:

TEXT

```
7
Parent field
Parent block
Child field
Child block
9
```

`Child.CONSTANT` is a `static final` primitive initialized by a constant expression. Its value can be embedded in the caller, so reading it does not trigger `Child` initialization. Reading `Child.boxed` is an active use of `Child`; before a class initializes, its superclass initializes. Within each class, static field initializers and static blocks execute in textual order. Therefore the parent field and block precede the child field and block.

The JVM may have loaded or verified either nested class earlier, and HotSpot may interpret or compile methods at any appropriate time. Those details do not change the specified initialization output. In a real system, avoid important side effects in static initialization: failure produces `ExceptionInInitializerError`, and later active use can fail because initialization did not complete.

Self-check. You should be able to:

- distinguish load, link, and initialize;
- identify a compile-time constant without running the code;
- state superclass-before-subclass initialization; and
- avoid claiming when JIT compilation occurs.

E.3 Language exercise: pass-by-value and aliasing

Problem. Explain why `swap` fails to swap the caller's variables while `rename` changes the shared object. Then repair the design so the caller can obtain swapped values without hidden mutation.

JAVA (continued 1/2)

```java
public final class PassByValueExercise {
    static final class User {
        private String name;

        User(String name) {
            this.name = name;
        }

        void rename(String newName) {
            name = newName;
        }

        String name() {
            return name;
        }
    }

    record Pair<T>(T first, T second) {}

    static void swap(User left, User right) {
        User temporary = left;
        left = right;
        right = temporary;
    }
```

JAVA (continued 2/2)

```
    static void rename(User user) {
        user.rename("renamed");
    }

    static <T> Pair<T> swapped(T left, T right) {
        return new Pair<>(right, left);
    }

    public static void main(String[] args) {
        User first = new User("first");
        User second = new User("second");

        swap(first, second);
        System.out.println(first.name() + "," + second.name());

        rename(first);
        System.out.println(first.name());

        Pair<User> result = swapped(first, second);
        first = result.first();
        second = result.second();
        System.out.println(first.name() + "," + second.name());
    }
}
```

Hint. Draw caller variables and parameter variables as separate boxes. Copy references into parameter boxes. Reassignment changes one box; mutation follows a reference to an object.

Selected solution. Java always passes argument values. For reference expressions, the value is a reference. `swap` receives copies of the two references and only reassigns its local copies, so the caller prints `first,second`. `rename` follows its copied reference to the same `User`, so the caller observes `renamed`. Returning a `Pair` communicates new reference placement explicitly; the final line is `second,renamed`.

The phrase "objects are passed by reference" is misleading because it predicts that parameter reassignment can update caller variables. It cannot. The language does not expose the physical address representation of a reference.

Self-check. Your diagram should contain four variable boxes before `swap`: two caller variables and two parameters. You should predict every output and distinguish reassignment, mutation, and immutable-result design.

E.4 Generics exercise: a type-safe copy operation

Problem. Implement a method that copies values from a producer collection into a consumer collection. It must allow `List<Integer>` to be copied into `List<Number>` without raw types or casts.

JAVA

```java
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;

public final class GenericCopyExercise {
    static <T> void copy(Collection<? extends T> source,
            Collection<? super T> destination) {
        for (T value : source) {
            destination.add(value);
        }
    }

    public static void main(String[] args) {
        List<Integer> integers = List.of(1, 2, 3);
        List<Number> numbers = new ArrayList<>();
        copy(integers, numbers);
        System.out.println(numbers);
    }
}
```

Hint. Use PECS: producer extends, consumer super. Ask what can be safely read and what can be safely written.

Selected solution. `source` has an unknown element type that is a subtype of `T`, so every value read from `? extends T` is assignable to `T`. `destination` can consume `T`, so `? super T` is safe for adding a `T`. Reading an arbitrary value from the destination only yields `Object` because its actual element type might be any supertype of `T`.

`List<Integer>` is not a subtype of `List<Number>`; generic types are invariant. If it were, a caller could add a `Double` through the `List<Number>` view and corrupt the integer list. Wildcards express use-site variance without permitting that write.

Self-check. Compile calls for `Integer -> Number`, `Integer -> Object`, and `Number -> Object`. Confirm that `Number -> Integer` is rejected. Explain the rejection without saying "the compiler is confused."

E.5 Collections exercise: stable frequency ranking

Problem. Count words case-insensitively and return entries ordered by descending frequency, then ascending normalized word. Do not depend on `HashMap` encounter order.

JAVA

```java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

public final class FrequencyRankingExercise {
    record Count(String word, int occurrences) {}

    static List<Count> rank(List<String> words) {
        Map<String, Integer> counts = new HashMap<>();
        for (String word : words) {
            if (word == null) {
                throw new IllegalArgumentException("null word");
            }
            String normalized = word.toLowerCase(Locale.ROOT);
            counts.merge(normalized, 1, Integer::sum);
        }

        ArrayList<Count> result = new ArrayList<>(counts.size());
        counts.forEach((word, count) -> result.add(new Count(word, count)));
        result.sort(Comparator.comparingInt(Count::occurrences).reversed()
                .thenComparing(Count::word));
        return List.copyOf(result);
    }

    public static void main(String[] args) {
        System.out.println(rank(List.of("Java", "map", "JAVA", "Map", "java")));
    }
}
```

Hint. Separate aggregation order from presentation order. Add a deterministic tie-breaker.

Selected solution. The hash map provides expected constant-time aggregation. The comparator completely defines presentation: higher counts first, then lexical word. The result is `[Count[word=java, occurrences=3], Count[word=map, occurrences=2]]`. `Locale.ROOT` avoids environment-dependent case mapping for protocol-like tokens.

Let `C` be the total character count and `u` the number of unique normalized words. Expected aggregation is `O(C)` because normalization, hashing, and equality inspect characters. Result creation is `O(u)`. Sorting performs `O(u log u)` comparisons, with lexical comparison cost proportional to the compared prefixes; it is only `O(u log u)` under a constant-length-word assumption. Space is `O(C)` in the worst case for retained normalized keys. If words are human-language text, normalization and collation require a domain-specific Unicode/locale policy beyond lowercase.

Self-check. Test empty input, ties, repeated case variants, and a null. Explain why stable sorting alone would not make tied output deterministic if the input to the sort came from an unordered map.

E.6 Collections design exercise: bounded LRU cache

Problem. Implement a small least-recently-used cache. State what the implementation does not guarantee.

JAVA (continued 1/2)

```
import java.util.LinkedHashMap;
import java.util.Map;

public final class LruCacheExercise<K, V> {
    private final int capacity;
    private final LinkedHashMap<K, V> entries;

    public LruCacheExercise(int capacity) {
        if (capacity < 1) {
            throw new IllegalArgumentException("capacity must be positive");
        }
        this.capacity = capacity;
        this.entries = new LinkedHashMap<>(16, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
                return size() > LruCacheExercise.this.capacity;
            }
        };
    }

    public V get(K key) {
```

JAVA (continued 2/2)

```
        return entries.get(key);
    }

    public void put(K key, V value) {
        entries.put(key, value);
    }

    public int size() {
        return entries.size();
    }

    public static void main(String[] args) {
        LruCacheExercise<String, Integer> cache = new LruCacheExercise<>(2);
        cache.put("a", 1);
        cache.put("b", 2);
        cache.get("a");
        cache.put("c", 3);
        System.out.println(cache.get("a")); // 1
        System.out.println(cache.get("b")); // null
    }
}
```

Hint. `LinkedHashMap` can maintain access order. Evict only after insertion makes size exceed capacity.

Selected solution. Accessing `a` moves it behind `b` in access order. Adding `c` makes `b` eldest, so the override removes `b`. Basic map operations are expected `O(1)`.

This is not thread-safe, does not distinguish an absent key from a cached null, uses entry count rather than byte weight, has no expiration, does not load values atomically, and is not distributed. Production cache

design needs an admission/eviction policy, metrics, concurrency, stampede control, and failure semantics. `LinkedHashMap` access-order behavior is an API contract; its node layout is not.

Self-check. Verify recency changes on both `get` and replacement. Explain how you would add synchronization without returning iterators that escape the lock. Define whether null keys/values are valid.

E.7 Concurrency exercise: safe lazy initialization

Problem. Replace a racy lazy singleton without explicit locking on every read. Explain its happens-before argument.

JAVA

```java
public final class LazyInitializationExercise {
    private LazyInitializationExercise() {}

    private static final class Holder {
        static final LazyInitializationExercise INSTANCE =
                new LazyInitializationExercise();
    }

    public static LazyInitializationExercise instance() {
        return Holder.INSTANCE;
    }

    public static void main(String[] args) throws InterruptedException {
        LazyInitializationExercise[] observed = new LazyInitializationExercise[2];
        Thread first = new Thread(() -> observed[0] = instance());
        Thread second = new Thread(() -> observed[1] = instance());
        first.start();
        second.start();
        first.join();
        second.join();
        System.out.println(observed[0] == observed[1]);
    }
}
```

Hint. Class initialization is synchronized by the JVM and happens-before later use of that class by any thread.

Selected solution. `Holder` is not initialized merely because the outer class initializes. The first active read of `Holder.INSTANCE` triggers `Holder` initialization. The initialization procedure permits one thread to perform it while other threads wait, and successful class initialization happens-before subsequent active use. Final-field semantics add safety for immutable constructed state, but the key publication edge here is class initialization.

The program prints `true`. `join` also gives the main thread a happens-before edge from each worker's completed actions, so the array reads are visible. Without `join` or another synchronization edge, reading the array from main would race.

An enum singleton is another simple solution when its serialization and initialization semantics fit. Double-checked locking requires a `volatile` field and a correct local/read pattern; the holder idiom is harder to get wrong.

Self-check. Name both happens-before edges: class initialization to active use, and worker actions to successful `join` return. Explain why "the object was constructed first" is not alone a memory-visibility proof.

E.8 Concurrency exercise: bounded task fan-out and cancellation

Problem. Run several tasks with a total timeout, cancel unfinished work, preserve interruption, and avoid leaking the executor.

JAVA (continued 1/2)

```
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;

public final class BoundedFanOutExercise {
    static <T> List<T> run(List<? extends Callable<T>> tasks,
            int parallelism, Duration timeout) throws Exception {
        if (parallelism < 1 || timeout.isNegative() || timeout.isZero()) {
            throw new IllegalArgumentException("invalid limits");
        }

        ExecutorService executor = Executors.newFixedThreadPool(parallelism);
        try {
            List<Future<T>> futures;
            try {
                futures = executor.invokeAll(
                        tasks, timeout.toNanos(), TimeUnit.NANOSECONDS);
            } catch (InterruptedException interrupted) {
                Thread.currentThread().interrupt();
                throw interrupted;
            }

            ArrayList<T> results = new ArrayList<>(futures.size());
            for (Future<T> future : futures) {
```

JAVA (continued 2/2)

```
                if (future.isCancelled()) {
                    throw new TimeoutException("fan-out deadline exceeded");
                }
                try {
                    results.add(future.get());
                } catch (ExecutionException failed) {
                    Throwable cause = failed.getCause();
                    if (cause instanceof Exception exception) {
                        throw exception;
                    }
                    throw (Error) cause;
                } catch (InterruptedException interrupted) {
                    Thread.currentThread().interrupt();
                    throw interrupted;
                }
            }
            return List.copyOf(results);
        } finally {
            executor.shutdownNow();
        }
    }

    public static void main(String[] args) throws Exception {
        List<Callable<String>> tasks = List.of(
                () -> "alpha",
                () -> "beta",
                () -> "gamma");
        System.out.println(run(tasks, 2, Duration.ofSeconds(1)));
    }
}
```

Hint. Use the timed bulk operation, but still inspect cancellation. Treat interrupt as cancellation, not a retryable ordinary exception.

Selected solution. `invokeAll` submits all tasks, waits no longer than the timeout, and cancels unfinished futures on return. A fixed pool bounds simultaneous work, not the submitted task list; callers must also bound list size. Results retain input order because the returned future list corresponds to task order, not completion order.

`shutdownNow` requests interruption but cannot force code that ignores interruption to stop. The task contract must honor cancellation and bound its own I/O. Before timed `invokeAll` returns, each task has either completed or been canceled because the timeout expired; throwing while inspecting the first failed future does not undo work that already completed. A production design must decide fail-fast versus collect-all outcomes, attach multiple failures if needed, and use bounded `awaitTermination` handling when executor termination matters to lifecycle safety.

Java 21 virtual threads can simplify blocking task-per-request code, but scarce downstream connections still require a semaphore, pool, or admission bound. Virtual threads remove thread scarcity, not resource limits.

Self-check. Test one slow task, one throwing task, and caller interruption. Confirm no test waits forever. State which order the returned results use and why cancellation is cooperative.

E.9 Performance exercise: repair a misleading benchmark

Problem. A candidate presents this result as proof that string parsing takes two nanoseconds:

JAVA

```java
long start = System.nanoTime();
for (int i = 0; i < 100_000_000; i++) {
    Integer.parseInt("123");
}
long elapsed = System.nanoTime() - start;
System.out.println(elapsed / 100_000_000.0);
```

Identify at least six validity problems and design a JMH experiment.

Hint. Ask whether the result is observable, the input is constant, compilation is warm, process history is independent, setup matches production, and uncertainty is reported.

Selected solution. Problems include:

1. The result is unused, so the optimizer can eliminate work.
2. The string is constant, so parsing can be folded or specialized.
3. One process mixes warmup and measurement.
4. One aggregate timing reports no iteration/fork variance.
5. The loop and division are part of a homemade harness.
6. There is no baseline for harness overhead.
7. One tiny valid input does not represent lengths, signs, failures, or varied digits.
8. Environment, JDK, flags, CPU limits, and thermal/background state are missing.
9. A microbenchmark does not establish endpoint impact.

A defensible JMH design uses `@Param` for representative strings prepared in trial state, returns the parsed value or consumes it in a `Blackhole`, uses several warmup and measurement iterations in multiple forks, and reports configuration and uncertainty. Invalid-input parsing should be a separate benchmark because exception construction changes the operation. Add allocation or compiler profiling in separate runs, then confirm through a production profile that parsing is hot enough to matter.

Do not make input unpredictability artificial. If production repeatedly parses a small fixed vocabulary, model that vocabulary. If values vary per request, prepare an array of varied inputs and advance an index without putting random generation in the timed operation.

Self-check. Your design must state the exact question, mode, inputs, state scope, setup level, result consumption, forks, warmup, measurement, and the higher-level validation needed.

E.10 DSA exercise: longest substring without repeating characters

Problem. Given an ASCII string, return the length of its longest substring containing no repeated character. Target `O(n)` time.

JAVA

```java
import java.util.Arrays;

public final class LongestUniqueWindowExercise {
    static int longestUnique(String text) {
        if (text == null) {
            throw new IllegalArgumentException("text is required");
        }
        int[] lastSeen = new int[128];
        Arrays.fill(lastSeen, -1);

        int left = 0;
        int best = 0;
        for (int right = 0; right < text.length(); right++) {
            char current = text.charAt(right);
            if (current >= 128) {
                throw new IllegalArgumentException("ASCII input required");
            }
            left = Math.max(left, lastSeen[current] + 1);
            lastSeen[current] = right;
            best = Math.max(best, right - left + 1);
        }
        return best;
    }

    public static void main(String[] args) {
        System.out.println(longestUnique("abba"));   // 2
        System.out.println(longestUnique("abcabcbb")); // 3
    }
}
```

Hint. Maintain a half-open or closed window invariant and jump the left boundary past the previous occurrence. Never move left backward.

Selected solution. Before processing `right`, the window from `left` through `right - 1` contains no duplicate. If the new character was seen inside that window, `lastSeen[current] + 1` moves `left` past it. `Math.max` prevents an older occurrence outside the window from moving `left` backward.

For `abba`, windows evolve as `a`, `ab`, then the second `b` moves left from 0 to 2, producing `b`; the final `a` has an old index 0 outside the current window, so left remains 2 and `ba` has length 2.

Each index advances once and table operations are constant, so time is `O(n)` and auxiliary space is `O(1)` for the fixed ASCII alphabet. Java `char` represents UTF-16 code units, not every Unicode code point. A Unicode-code-point contract should iterate `text.codePoints()` and use a map or appropriately sized structure.

Self-check. Test empty, one-character, all-equal, all-unique, and `abba`. State the window invariant before writing complexity.

E.11 DSA exercise: deterministic topological ordering

Problem. Given directed edges `prerequisite -> dependent`, return a deterministic topological order or reject a cycle.

JAVA (continued 1/2)

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.Set;

public final class TopologicalOrderExercise {
    record Edge(String prerequisite, String dependent) {}

    static List<String> order(Set<String> nodes, List<Edge> edges) {
        Map<String, List<String>> outgoing = new HashMap<>();
        Map<String, Integer> indegree = new HashMap<>();
        for (String node : nodes) {
            outgoing.put(node, new ArrayList<>());
            indegree.put(node, 0);
        }

        Set<Edge> uniqueEdges = new HashSet<>();
        for (Edge edge : edges) {
            if (!nodes.contains(edge.prerequisite())
                    || !nodes.contains(edge.dependent())) {
                throw new IllegalArgumentException("edge contains unknown node");
            }
            if (uniqueEdges.add(edge)) {
                outgoing.get(edge.prerequisite()).add(edge.dependent());
                indegree.merge(edge.dependent(), 1, Integer::sum);
            }
        }
```

JAVA (continued 2/2)

```
        PriorityQueue<String> ready = new PriorityQueue<>();
        indegree.forEach((node, degree) -> {
            if (degree == 0) ready.offer(node);
        });

        ArrayList<String> result = new ArrayList<>(nodes.size());
        while (!ready.isEmpty()) {
            String node = ready.poll();
            result.add(node);
            for (String dependent : outgoing.get(node)) {
                int remaining = indegree.merge(dependent, -1, Integer::sum);
                if (remaining == 0) ready.offer(dependent);
            }
        }

        if (result.size() != nodes.size()) {
            throw new IllegalArgumentException("graph contains a cycle");
        }
        return List.copyOf(result);
    }

    public static void main(String[] args) {
        Set<String> nodes = Set.of("compile", "test", "package", "deploy");
        List<Edge> edges = List.of(
                new Edge("compile", "test"),
                new Edge("test", "package"),
                new Edge("package", "deploy"));
        System.out.println(order(nodes, edges));
    }
}
```

Hint. Kahn's algorithm repeatedly removes a zero-indegree node. If nodes remain but none is ready, those remaining dependencies contain a cycle.

Selected solution. Indegree counts unmet prerequisites. Removing a ready node conceptually removes all its outgoing edges; a dependent becomes ready exactly when its count reaches zero. The priority queue provides lexical determinism among simultaneously ready nodes. Duplicate edges are removed so they do not inflate indegree.

With `V` nodes and `E` unique edges, map construction and edge processing are expected `O(V + E)`. Each node enters the heap once, adding `O(V log V)` for deterministic selection; total is `O(E + V log V)` and space `O(V + E)`. A simple deque gives `O(V + E)` but its order depends on insertion/encounter policy.

Self-check. Test disconnected nodes, two valid choices, duplicate edges, a self-loop, a longer cycle, and an unknown endpoint. Prove that every emitted node has all prerequisites emitted first.

E.12 DSA exercise: top-k values without sorting everything

Problem. Return the largest `k` integers in descending order. Use `O(k)` auxiliary storage when `k` is much smaller than `n`.

JAVA

```java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

public final class TopKExercise {
    static List<Integer> largest(List<Integer> values, int k) {
        if (k < 0 || k > values.size()) {
            throw new IllegalArgumentException("k out of range");
        }
        PriorityQueue<Integer> retained = new PriorityQueue<>(Math.max(1, k));
        for (Integer value : values) {
            if (value == null) throw new IllegalArgumentException("null value");
            if (retained.size() < k) {
                retained.offer(value);
            } else if (k > 0 && value > retained.peek()) {
                retained.poll();
                retained.offer(value);
            }
        }
        ArrayList<Integer> result = new ArrayList<>(retained);
        result.sort(Comparator.reverseOrder());
        return List.copyOf(result);
    }

    public static void main(String[] args) {
        System.out.println(largest(List.of(4, 1, 9, 2, 9, 7), 3));
    }
}
```

Hint. Keep the worst currently retained answer at the heap head. It can be replaced in logarithmic time.

Selected solution. A min-heap of size `k` stores the largest values seen so far. Once full, a value no greater than the root cannot belong to a strictly better top-k multiset; a larger value replaces the root. Duplicate values are retained because the problem asks for values, not unique values.

For `1 <= k <= n`, scanning costs `O(n log k)`, sorting the result costs `O(k log k)`, and auxiliary space is `O(k)`. For `k = 0`, this implementation still validates all inputs and therefore costs `O(n)` time and `O(1)` auxiliary space. For `k` close to `n`, sorting a copy may be simpler and faster. Priority-queue iteration is not sorted, so the final explicit sort is necessary.

Self-check. Test `k = 0`, `k = n`, duplicates, negative values, and invalid `k`. Explain why arbitrary priority-queue iteration cannot be returned directly.

E.13 Backend design exercise: idempotent order plus outbox

Problem. Design `POST /orders` so a client can safely retry after a timeout. The service must create one logical order and eventually emit `OrderCreated` without a distributed transaction.

Hint. Separate client command identity, local atomic state, relay delivery, and consumer effect identity.

Selected solution. Require an idempotency key scoped to authenticated customer and operation. In one database transaction:

1. use a database-tested atomic insert-if-absent or upsert for an idempotency row containing `(customer, operation, key)`, a canonical request fingerprint, status `IN_PROGRESS`, and a generated order ID under a unique constraint;
2. when that operation reports an existing row, lock/read it where the database permits the read in the same transaction; if a uniqueness error aborts the transaction, roll it back and read in a new transaction. Reject a different fingerprint, return the stored completed response, or report/internally coordinate an in-progress attempt;
3. insert the order using the reserved order ID;
4. insert a versioned outbox event with a stable event ID and aggregate ID;
5. store the completed response/status in the idempotency row; and
6. commit.

A relay claims committed outbox rows, publishes, and records progress. If it crashes after broker acceptance but before progress commit, it republishes. Consumers therefore record event ID in an inbox or use a naturally idempotent conditional update in the same transaction as their effect. This is recoverable at-least-once delivery; it does not promise one physical delivery.

The HTTP request has an end-to-end deadline. A timeout is an unknown outcome, so the client retries the same key or queries order status. The key record's retention exceeds the maximum retry window. Cleanup never removes an in-progress record without reconciliation. Metrics include key conflicts, in-progress age, outbox age, publish attempts, poison events, and consumer duplicates.

Security rules include binding the key to customer identity, limiting key length and cardinality, comparing request fingerprints, redacting payloads, and authorizing status lookup. Reliability rules include a unique database constraint as the concurrent truth, bounded relay batches, leases, backoff, and dead-letter/repair workflow.

Self-check. Your answer must identify these crash windows: before commit, after commit before response, after publish before relay acknowledgement, and during consumer effect. For each, state the durable record that permits recovery and the stable identity that prevents duplicate logical effects.

E.14 Low-level design exercise: notification delivery

Problem. Design a notification component supporting email and SMS today, another channel later, templates, user preferences, retries, and audit. Do not build a pattern catalog for its own sake.

Hint. Separate application orchestration, policy, rendering, channel adapter, durable command state, and delivery attempts.

Selected solution. A compact domain API might be:

JAVA (continued 1/2)

```java
import java.time.Instant;
import java.util.Locale;
import java.util.Map;

enum Channel { EMAIL, SMS }

record NotificationCommand(String commandId, String userId,
        String templateId, Map<String, String> parameters, Instant createdAt) {
    public NotificationCommand {
        parameters = Map.copyOf(parameters);
    }
}

record Recipient(String address, Locale locale) {}
record RenderedMessage(String subject, String body) {}
record DeliveryResult(String providerMessageId, Instant acceptedAt) {}

interface PreferencePolicy {
    java.util.List<Channel> allowedChannels(String userId);
}

interface TemplateRenderer {
```

JAVA (continued 2/2)

```java
    RenderedMessage render(String templateId, Locale locale,
            Map<String, String> parameters);
}

interface ChannelSender {
    Channel channel();
    DeliveryResult send(String idempotencyKey, Recipient recipient,
            RenderedMessage message) throws DeliveryException;
}

final class DeliveryException extends Exception {
    private final boolean retryable;

    DeliveryException(String message, boolean retryable) {
        super(message);
        this.retryable = retryable;
    }

    boolean retryable() {
        return retryable;
    }
}
```

`NotificationService` validates and durably stores the command before asynchronous processing. `PreferencePolicy` is policy; `TemplateRenderer` owns safe template expansion; each `ChannelSender` is a Strategy and vendor adapter. A factory or map selects a sender by its declared channel. Metrics can be a Decorator, but retry belongs in orchestration because it needs durable attempt state and idempotency.

The command ID is the logical idempotency identity. A delivery table records channel, recipient version, template version, attempt, next eligible time, status, and safe provider response. Workers claim bounded batches with a lease. Retryable errors receive jittered backoff under an attempt/deadline policy; permanent address or authorization errors do not retry. Provider timeouts are unknown outcomes, so the sender passes a stable idempotency key when supported or reconciles provider status.

Preferences and destination addresses can change. Decide whether the command snapshots them at acceptance or resolves them at delivery; that business rule affects audit and privacy. Templates require versioning so a retry does not silently change content. Audit records exclude secrets and sensitive body content unless retention policy explicitly permits it.

Adding push notification introduces one `Channel`, sender, address policy, and integration tests without changing existing senders. This is Open/Closed Principle applied at an observed variation boundary. The system still needs rate limits per tenant/channel, provider bulkheads, queue bounds, graceful shutdown, and deletion/redaction workflows.

Self-check. State ownership, lifecycle, idempotency, retry classification, timeout outcome, template/preference version, provider isolation, audit sensitivity, and extension path. If your design sends directly in the request transaction, explain how it recovers from every crash window.

E.15 Final self-assessment rubric

Score each selected solution from 0 to 2 on each dimension:

A score below 10 suggests revisiting the chapter before adding more problems. A score of 12 or more is interview-ready only if you can reproduce the reasoning without seeing the solution. Practice explaining each answer in three layers: a 30-second summary, a two-minute model, and a detailed defense with code and edge cases.

Dimension	0	1	2
Contract	assumptions missing	partial contract	inputs, outputs, failure, ownership explicit
Invariant	absent	named but not used	drives representation and proof
Correctness	example only	plausible explanation	general argument plus dry run
Complexity	missing/wrong	time only	time, space, assumptions, constants discussed
Edge cases	none	common cases	boundaries, invalid input, overflow/null/concurrency
Production	ignored	generic note	limits, observability, lifecycle, failure policy
Communication	code dump	understandable	answer-first, structured, trade-offs explicit

SDE-2 CORE CHAPTER

Chapter 5 - Glossary

F.1 Reading the glossary

Definitions describe the Java 21 platform unless a version is named. "HotSpot" identifies common OpenJDK implementation behavior rather than a Java specification guarantee. Chapter references point to the principal treatment; many concepts recur throughout the book.

A

- **Abstract class:** A class declared `abstract` that cannot be instantiated directly and may combine implemented methods, state, constructors, and abstract methods for subclasses. See Master Chapter 18.
- **Abstraction:** A model exposing behavior that clients need while hiding representation or mechanism. Good abstractions state invariants, ownership, failure, and performance boundaries. See Master Chapter 49.
- **Access modifier:** `public`, `protected`, or `private`. Where the declaration rules permit it, omitting an access modifier gives package access; interface members and implicitly declared constructors have additional rules. Modifiers control source-level member/type accessibility, not network authorization or runtime data secrecy. See Master Chapter 16.
- **ACID:** Atomicity, consistency, isolation, and durability: goals used to reason about database transactions. Exact isolation and durability depend on the database and configuration. See Master Chapter 50.
- **Active use:** An action that can trigger class or interface initialization, such as selected static field access, static method invocation, instance creation, or reflective request. See Master Chapter 5.
- **Adapter:** A design pattern translating one contract into another, commonly isolating a vendor API from domain policy. A good adapter prevents vendor types from leaking inward. See Master Chapter 49.
- **Allocation rate:** Bytes or objects allocated per time. High allocation can drive GC work without implying a retention leak. Correlate it with live-set and latency evidence. See Master Chapters 39 and 41.
- **Amortized analysis:** Averaging total cost over an operation sequence. Dynamic-array append is amortized constant time even though an individual resize copies many references. See Master Chapters 26 and 42.
- **Annotation:** Metadata attached to declarations, types, or other supported locations. Retention and target control where it is available; behavior comes from compilers, runtimes, or frameworks. See Master Chapter 21.
- **API contract:** Observable promises of a type or operation: valid inputs, results, failures, side effects, ordering, mutation, blocking, ownership, and thread safety. See Master Chapter 49.
- **Array covariance:** Java permits a `Sub[]` where a `Super[]` is expected. Runtime store checks preserve type safety and can throw `ArrayStoreException`; generics are invariant. See Master Chapters 15 and 22.
- **Atomic operation:** An operation observed as indivisible under its stated concurrency model. Atomicity of one method does not make a multi-call invariant atomic. See Master Chapters 35 and 37.
- **Atomicity:** All-or-nothing behavior within a defined boundary. CPU atomics, Java atomic classes, and database transactions have different scopes and guarantees. See Master Chapters 35 and 50.
- **Autoboxing:** Compiler conversion from a primitive to its wrapper, such as `int` to `Integer`. It can add allocation, null, identity, and overload-selection surprises. See Master Chapter 12.

B

- **Backpressure:** A policy that slows, rejects, blocks, sheds, or durably spills production when consumers lack capacity. It prevents an unbounded queue from hiding overload. See Master Chapters 36 and 52.
- **Balanced tree:** A search tree with structural constraints keeping height logarithmic. Java's API need not prescribe the exact balancing algorithm of every ordered collection. See Master Chapters 28 and 45.
- **Behavioral subtyping:** Requirement that a subtype preserve the supertype's preconditions, postconditions, invariants, failures, and relevant timing/side-effect behavior. This is central to LSP. See Master Chapter 49.
- **Big-O notation:** An asymptotic upper-bound language that suppresses constant factors and lower-order terms. Always state input variable, operation, and assumptions. See Master Chapter 42.
- **Binary heap:** A complete tree, usually array-backed, satisfying a parent-child priority invariant. It exposes an extremum efficiently but is not globally sorted. See Master Chapters 29 and 45.
- **Binary search tree:** A tree whose left and right subtrees are ordered relative to each node under a comparator. Without balancing, height can become linear. See Master Chapters 28 and 45.
- **Blocking:** Waiting while a thread cannot make current progress, for example on I/O, a lock, or a queue. Blocking semantics need timeout, interruption, and capacity policy. See Master Chapters 33 and 37.
- **Boxing:** Explicit or implicit creation/use of a wrapper representation for a primitive value. Primitive streams and specialized collections/arrays can avoid some boxing overhead. See Master Chapters 12 and 31.

C

- **Cache locality:** Performance benefit from accessing nearby or recently used memory. Array-backed collections generally have better locality than node-heavy linked structures. See Master Chapters 26 and 39.
- **Capacity:** Allocated storage or admitted workload limit, distinct from current logical size. Explicit capacity is also a reliability boundary for queues, pools, and caches. See Master Chapters 26, 29, and 52.
- **CAS:** Compare-and-set, an atomic conditional update that succeeds only if the observed value still equals an expected value. Algorithms must handle failure and possible ABA concerns. See Master Chapter 35.
- **Checked exception:** A `Throwable` subtype that is neither a `RuntimeException` subtype nor an `Error` subtype. Java's compile-time checking generally requires callers to catch or declare checked exceptions. They should represent meaningful recoverable boundaries. See Master Chapter 20.
- **Class:** A Java reference-type declaration defining members, constructors, inheritance, and implementation. At runtime a class is associated with its defining loader. See Master Chapters 16 and 5.
- **Class file:** The binary format containing JVM instructions, constants, metadata, and verification information for a class or interface. It is specified by the JVMS. See Master Chapter 3.
- **Class initialization:** Execution of static field initializers and static blocks under the language's initialization procedure. Successful initialization has important inter-thread visibility guarantees. See Master Chapter 5.
- **Class loader:** Runtime component that creates a `Class` from binary data and establishes a namespace. The same binary name under different defining loaders denotes different runtime types. See Master Chapter 5.
- **Class path:** Ordered set of locations searched by class-loading tools in class-path mode. Duplicate classes and ordering can make resolution fragile; modules provide another model. See Master Chapters 2 and 51.
- **Class variable:** A `static` field associated with a class rather than each instance. Shared mutability requires an explicit concurrency and lifecycle policy. See Master Chapters 16 and 33.
- **Class-loader leak:** Retention of an otherwise obsolete loader through threads, statics, registries, drivers, or callbacks, thereby retaining all classes and related metadata it defines. See Master Chapter 41.
- **Closure:** Function-like behavior together with captured lexical values. A Java lambda can capture only final or effectively final local variables, though captured objects may be mutable. See Master Chapter 23.
- **Code cache:** HotSpot native memory storing generated machine code and related metadata. Its organization and tuning are implementation-specific. See Master Chapters 10 and 40.
- **Cohesion:** Degree to which a component's responsibilities belong together and change for related reasons. High cohesion makes invariants and ownership easier to locate. See Master Chapter 49.
- **Collision:** In hashing, distinct unequal keys mapping to the same hash region. Correct tables resolve collisions using equality; excessive collisions degrade performance. See Master Chapter 27.
- **Comparable:** Interface defining a type's natural ordering through `compareTo`. Natural order should normally be consistent with `equals`, especially for sorted collection use. See Master Chapter 30.

- **Comparator:** Object defining an external ordering through `compare`. It must satisfy sign, transitivity, and zero-consistency rules; comparison zero defines uniqueness in tree collections. See Master Chapter 30.
- **Composition:** Building behavior by owning and delegating to collaborators. It usually exposes fewer extension hazards than inheritance and supports Strategy or Decorator designs. See Master Chapter 49.
- **Concurrency:** Multiple tasks making progress during overlapping periods, whether or not they execute simultaneously. Correctness requires ordering, ownership, cancellation, and capacity reasoning. See Part V.
- **Concurrent collection:** Collection designed with specified thread-safe operations and traversal semantics. Its individual method safety does not automatically make a compound workflow atomic. See Master Chapter 37.
- **Constant pool:** See runtime constant pool. A class file also contains a per-class-file constant pool used to encode symbolic and literal information. See Master Chapters 3 and 4.
- **Constructor:** Special class member that initializes a newly created instance after allocation and superclass construction steps. It is not inherited and has no return type. See Master Chapter 16.
- **Contention:** Multiple threads competing for a limited resource such as a lock, CPU, connection, or cache line. It can reduce throughput and increase tail latency. See Master Chapters 34 and 39.
- **Covariance:** Type relationship preserving direction, such as `? extends T` for a producer view. Java arrays are covariant; ordinary generic types are invariant. See Master Chapter 22.
- **Critical section:** Code region whose shared-state invariant requires controlled access, commonly through a lock or atomic protocol. Keep it correct first, then minimize held time. See Master Chapter 34.

D

- **Data race:** Conflicting accesses to shared memory by different threads, at least one a write, without sufficient happens-before ordering. Race-free design is prerequisite to simple visibility reasoning. See Master Chapter 11.
- **Deadlock:** Cycle of waiting in which each participant requires a resource held by another and none can progress. Prevention can impose lock ordering or avoid hold-and-wait. See Master Chapter 38.
- **Defensive copy:** Independent copy made to prevent caller or callee aliases from mutating owned collection/array structure. A shallow copy still shares element objects. See Master Chapters 19 and 49.
- **Dependency injection:** Supplying collaborators from outside a component rather than constructing global/concrete dependencies internally. It improves explicitness and test seams when boundaries are meaningful. See Master Chapter 49.
- **Dependency inversion:** High-level policy depends on an abstraction it needs; low-level mechanisms implement that abstraction. An interface mirroring one vendor SDK does not necessarily invert policy. See Master Chapter 49.
- **Deserialization:** Reconstruction of data or objects from an external representation. It is a trust boundary requiring schema, type, size, depth, and code-execution controls. See Master Chapters 32 and 52.
- **Direct buffer:** NIO buffer whose storage is outside the ordinary heap-array model and intended for efficient native I/O. Allocation, cleanup, and accounting are JVM/platform-sensitive. See Master Chapter 32.
- **Dominator:** In heap-graph analysis, an object through which every root path to another object passes. Dominators help locate retention owners. See Master Chapter 41.
- **Double-checked locking:** Lazy-initialization pattern checking before and after acquiring a lock. In Java, the published field must be `volatile` and the implementation must preserve the protocol. See Master Chapter 35.
- **Downstream collector:** Collector applied inside another collector, such as counting values inside each `groupingBy` bucket. It enables declarative multi-level reductions. See Master Chapter 31.
- **DTO:** Data transfer object designed for a boundary such as HTTP, events, or persistence mapping. It should carry an explicit version/validation contract rather than expose internal entities. See Master Chapter 50.
- **Durability:** Degree to which committed state survives failures under a storage system's contract. Java `flush` or object reachability does not itself imply durable storage. See Master Chapters 32 and 50.
- **Dynamic dispatch:** Runtime selection of an overridden instance method based on the receiver's runtime class. Static, private, and constructor behavior follows different rules. See Master Chapter 17.
- **Dynamic programming:** Algorithm technique storing solutions to overlapping subproblems under a recurrence and state definition. Correct state and transition derivation precede table optimization. See Master Chapter 47.

E

- **Encapsulation:** Protecting representation and invariants through controlled operations. Private fields alone are insufficient if mutable aliases or invalid setters escape. See Master Chapters 16 and 49.
- **Encounter order:** Logical order in which a collection or stream presents elements. It can be defined, absent, or deliberately relaxed; it is distinct from sorted order. See Master Chapters 25 and 31.
- **Enum:** Special class with a fixed set of named instances. Enums can have fields, methods, and per-constant behavior and work efficiently with `EnumSet`/`EnumMap`. See Master Chapter 21.
- **Equality contract:** `equals` must be reflexive, symmetric, transitive, consistent, and false for null. Equal objects must have equal hash codes. See Master Chapter 19.
- **Erasure:** See type erasure.
- **Escape analysis:** JVM optimization analysis determining whether an object/reference escapes a scope, potentially enabling allocation or synchronization elimination. Results are HotSpot/version-dependent. See Master Chapters 7 and 39.
- **Exception:** Object representing abrupt completion. Checked/unchecked classification, causal chains, suppression, and recovery scope are API-design concerns. See Master Chapter 20.
- **Executor:** Abstraction separating task submission from scheduling/execution policy. Executor services also define lifecycle, rejection, queue, and shutdown behavior. See Master Chapter 36.

F

- **Fail-fast iterator:** Iterator that attempts to detect unsupported structural modification and throw `ConcurrentModificationException`. Detection is best-effort and is not synchronization. See Master Chapter 25.
- **Fairness:** Policy controlling which waiter obtains a resource. Fair locks/queues can reduce starvation but often add coordination cost and do not guarantee application-level fairness. See Master Chapter 34.
- **False sharing:** Performance interference when independent frequently written variables occupy the same hardware cache line. It is hardware/layout-sensitive and should be confirmed by measurement. See Master Chapter 39.
- **FIFO:** First in, first out removal policy. Queue ordering can still be affected by priorities, multiple consumers, retry insertion, or implementation semantics. See Master Chapter 29.
- **Final field:** Field assigned once by constructor/initializer rules. Correct construction gives special visibility guarantees for final state, but referenced mutable objects can still change. See Master Chapters 11 and 19.
- **ForkJoinPool:** Executor based on work-stealing queues, suited to recursively divisible CPU work. Blocking and use of the common pool require capacity awareness. See Master Chapter 36.
- **Functional interface:** A non-sealed interface whose inherited abstract methods, after excluding signatures matching public methods of `Object`, induce one function type under the JLS rules. It can be a target for lambdas and method references; default and static methods do not add abstract requirements. See Master Chapter 23.

G

- **G1:** Garbage-First collector, a region-based HotSpot collector and common default in mainstream Java 17/21 server configurations. It is an implementation choice, not Java semantics. See Master Chapter 9.
- **Garbage collection:** Automatic reclamation of storage for unreachable objects. Java does not mandate one collector, generation scheme, pause target, or immediate return of memory to the OS. See Master Chapter 9.
- **GC root:** Starting reference used in reachability analysis, such as selected thread, static, class-loader, JNI, or VM references. Root paths explain retention. See Master Chapter 41.
- **Generics:** Compile-time parameterization of types and methods. Java generics largely use erasure and are invariant unless wildcards express use-site variance. See Master Chapter 22.

H

- **Happens-before:** JMM ordering relation guaranteeing visibility and ordering between actions. Program order, locks, volatile, thread start/join, class initialization, and transitivity create important edges. See Master Chapter 11.
- **Hash code:** Integer used to choose candidate hash regions. Equal objects must have equal hash codes; unequal objects may collide. Key-relevant state must remain stable while stored. See Master Chapter 27.
- **Hash flooding:** Accidental or adversarial concentration of keys into collisions, increasing CPU cost. Current implementations may add resilience, but input bounds and sound hashes remain necessary. See Master Chapter 27.
- **HashMap:** General-purpose hash-table `Map` allowing one null key and null values, with expected constant-time basic operations under suitable hashing. It has no encounter-order guarantee. See Master Chapter 27.
- **Heap (data structure):** Priority structure satisfying a heap-order invariant between parents and descendants. A complete, array-backed binary heap is the common representation in Java priority-queue and top-k discussions. See binary heap and Master Chapters 29 and 45.
- **Heap (JVM):** Runtime data area from which class instances and arrays are allocated conceptually. Physical representation and collector layout are implementation-specific. See Master Chapter 6.
- **Heap pollution:** A parameterized variable refers to an object not compatible with its parameterized type, often through raw types, unchecked casts, arrays, or unsafe varargs. See Master Chapter 22.
- **HotSpot:** OpenJDK's widely used JVM implementation. Its JIT tiers, collectors, object layout, flags, and diagnostics are useful engineering details but not Java specification guarantees.

I

- **Idempotency:** Property that repeated execution under one logical identity produces the intended single logical effect. It is essential when timeouts leave remote outcomes unknown. See Master Chapters 50 and 52.
- **Identity:** Distinction between the same object reference and merely equal values. `==` tests reference identity for references; `equals` can define logical value equality. See Master Chapter 19.
- **Immutability:** Inability to change an object's observable state after construction. Final fields and unmodifiable collections help, but deep immutable graphs require controlling mutable referenced objects. See Master Chapter 19.
- **Inheritance:** Mechanism by which a class extends another class and inherits accessible members/behavior. Correct use requires a genuine behavioral subtype, not merely code reuse. See Master Chapter 17.
- **Interface:** Reference type declaring a contract and possibly default/static/private methods and constants. Interfaces support multiple capability inheritance but do not automatically guarantee substitutability. See Master Chapter 18.
- **Interrupt:** Cooperative thread cancellation/status mechanism. Blocking methods may throw `InterruptedException`; code should propagate or restore status unless it deliberately consumes cancellation. See Master Chapter 33.
- **Invariant:** Condition that remains true for every valid observable state of an object, algorithm, or system. It is the center of correctness proofs and API design. See Master Chapters 42 and 49.
- **Isolation:** Transaction property controlling observable interference among concurrent transactions. Named levels and anomalies have database-specific realization and must be integration-tested. See Master Chapter 50.
- **Iterator:** Stateful traversal cursor with `hasNext` and `next`, plus optional removal. Iterator ordering, mutation, consistency, and thread behavior come from the source contract. See Master Chapter 25.

J

- **JAR:** ZIP-based Java archive format containing classes, resources, and metadata. A JAR is packaging, not an isolation or dependency-resolution mechanism. See Master Chapters 2 and 51.
- **javac:** Reference Java compiler included with the JDK. It translates source according to selected language/release options; its diagnostics and optimization choices are tool-specific. See Master Chapter 3.
- **JDK:** Java Development Kit: a Java runtime plus development, packaging, documentation, and diagnostic tools. Distribution and included components vary by vendor. See Master Chapter 2.
- **JFR:** Java Flight Recorder, a JDK event-recording facility for timestamped runtime/application evidence. Event configuration and fields are version-sensitive. See Master Chapter 40.
- **JIT:** Just-in-time compilation of hot runtime code to machine code. Java permits but does not require a particular compiler, threshold, tier, or optimization. See Master Chapter 10.
- **JLS:** Java Language Specification, the normative definition of Java syntax, typing, evaluation, initialization, exceptions, and memory-model rules for a release.
- **JMC:** JDK Mission Control, a separately distributed analysis and monitoring application commonly used to inspect JFR recordings. Tool versions should match supported recording formats. See Master Chapter 40.
- **JNI:** Java Native Interface, a boundary for calling native code and interacting with JVM objects. It expands memory-safety, lifecycle, portability, and crash risk. See Master Chapters 4 and 52.
- **JVM:** Abstract Java Virtual Machine plus, colloquially, an implementation executing class files. Separate JVMS guarantees from HotSpot behavior. See Master Chapter 4.
- **JVMS:** Java Virtual Machine Specification, the normative definition of class-file format, runtime data areas, loading/linking/initialization, instructions, and verification for a release.

L

- **Lambda expression:** Java syntax producing an instance of a functional-interface target type. Capture and runtime translation do not imply a specified anonymous-class allocation. See Master Chapter 23.
- **Latency:** Elapsed time for an operation. It is a distribution; p50, p95, p99, maximum, errors, and load context convey more than an average. See Master Chapter 39.
- **Lazy evaluation:** Deferring work until a result is demanded. Stream intermediate operations are generally lazy, enabling fusion and short-circuiting but allowing stateful buffering. See Master Chapter 31.
- **Linearizability:** Concurrent-object correctness condition in which each completed operation appears to take effect at one instant between invocation and response, respecting real-time order. See Master Chapter 37.
- **Linking:** JVM process of verification, preparation, and optionally resolution after loading. Verification and preparation precede initialization, while symbolic-reference resolution may continue after initialization. See Master Chapter 5.
- **Live set:** Memory occupied by objects that remain reachable after an effective collection/observation point. A growing comparable live set suggests increasing intended or unwanted retention. See Master Chapter 41.
- **Livelock:** Threads continue taking actions in response to one another but make no useful progress. Randomized/backoff coordination or protocol redesign may resolve it. See Master Chapter 38.
- **Load factor:** Hash-table fullness parameter influencing resize threshold, empty buckets, and collision depth. Exact capacity policy belongs to the concrete implementation. See Master Chapter 27.
- **Lock:** Synchronization mechanism granting controlled access and creating memory-ordering edges. Intrinsic monitors and `Lock` implementations differ in API features. See Master Chapter 34.
- **LSP:** Liskov Substitution Principle: implementations must remain behaviorally usable wherever the abstraction is expected. See behavioral subtyping and Master Chapter 49.

M

- **Map:** Association from unique keys to values. `Map` is not a subtype of `Collection`; its key, value, and entry views bridge into collection APIs. See Master Chapter 25.
- **Memory leak:** Unwanted retention or unreleased resource that grows footprint or exhausts capacity. In Java heap analysis, the root issue is an unintended reachability path. See Master Chapter 41.
- **Memory model:** Rules describing legal inter-thread observations, ordering, data races, final fields, volatile, locks, and happens-before. Java's model is specified in JLS Master Chapter 17. See Master Chapter 11.
- **Metaspace:** HotSpot native-memory area commonly holding class metadata. It can grow through class loading and class-loader retention and is distinct from ordinary Java heap. See Master Chapters 6 and 41.
- **Method area:** JVMS shared runtime area storing per-class structures such as runtime constants and method/field data. Physical placement is not specified. See Master Chapter 6.
- **Method overloading:** Multiple methods share a name but have different parameter signatures; compile-time resolution selects among applicable methods. Return type alone cannot overload. See Master Chapter 14.
- **Method overriding:** Subclass or implementation supplies an instance method matching an inherited method contract, enabling dynamic dispatch. Visibility, return, and checked-exception rules constrain it. See Master Chapter 17.
- **Module:** Named unit in the Java Platform Module System declaring dependencies and exported/open packages. Modules improve reliable configuration and encapsulation but are not security sandboxes. See Master Chapter 2.
- **Monitor:** Intrinsic synchronization object associated conceptually with each object/class, used by `synchronized`, `wait`, `notify`, and `notifyAll`. See Master Chapter 34.
- **Mutable reduction:** Accumulation into a mutable container using a supplier, accumulator, combiner, and optional finisher, as modeled by stream collectors. See Master Chapter 31.

N

- **N+1 query:** One initial query followed by one query per result/item, causing linear database round trips. Batch/fetch design should preserve cardinality and memory bounds. See Master Chapter 50.
- **Natural ordering:** Canonical ordering defined by `Comparable`. Sorted maps/sets use it when no comparator is supplied; comparison zero controls their key/element uniqueness. See Master Chapters 28 and 30.
- **NIO:** Java APIs centered on buffers, channels, selectors, charsets, and modern filesystem paths. NIO does not mean every operation is nonblocking. See Master Chapter 32.
- **Non-interference:** Stream requirement that behavioral functions not improperly modify or depend on mutation of the source during pipeline execution. See Master Chapter 31.
- **Null:** The sole value of the null type, assignable to reference types but not primitive types. API null support is contract-specific; dereference throws `NullPointerException`. See Master Chapter 12.

O

- **Object:** Class instance or array at runtime. A reference variable holds a reference value, not the object inline as a language guarantee. See Master Chapters 7 and 16.
- **Object header:** HotSpot-specific metadata commonly associated with an object, such as class and locking/GC information. Exact bits and size depend on JVM/configuration. See Master Chapter 7.
- **Object monitor:** See monitor.
- **Optional:** Value-based container representing one non-null value or absence. Best used as a maybe-one return type, not universally as a field, parameter, or collection wrapper. See Master Chapter 31.
- **Outbox:** Durable table/record written atomically with domain changes and later relayed to messaging, addressing the database-plus-broker dual-write gap. See Master Chapter 50.
- **Overloading:** See method overloading.
- **Overriding:** See method overriding.

P

- **Parallel stream:** Stream pipeline eligible for partitioned execution, commonly using fork/join infrastructure. Correctness needs associative/stateless behavior; speed depends on source splitting and workload. See Master Chapter 31.
- **Parameterized type:** Generic type instantiated with type arguments, such as `List<String>`. It provides compile-time constraints but is mostly erased at runtime. See Master Chapter 22.
- **Pass-by-value:** Java argument passing copies the evaluated argument value into a parameter. For objects, the copied value is a reference; parameter reassignment does not affect caller variables. See Master Chapter 14.
- **Pattern matching:** Language features that test structure/type and introduce variables when a pattern matches, including `instanceof`, record patterns, and pattern `switch`. See Master Chapters 18 and 24.
- **Platform thread:** `Thread` typically mapped one-to-one to an operating-system thread for its lifetime. It is heavier and scarcer than a virtual thread. See Master Chapters 33 and 37.
- **Polymorphism:** One abstraction supports values with differing implementations, with behavior selected through overriding, interfaces, or patterns. Substitutability remains a contract requirement. See Master Chapter 17.
- **Prepared statement:** JDBC statement separating SQL structure from bound values. It mitigates value injection but cannot parameterize arbitrary identifiers or SQL fragments. See Master Chapter 50.
- **Primitive type:** One of Java's boolean or numeric non-reference types. Primitive values have defined ranges/semantics and cannot be null. See Master Chapter 12.
- **Priority queue:** Queue whose head is selected by ordering rather than arrival time. Java's `PriorityQueue` is not stable and its iterator is not sorted. See Master Chapter 29.
- **Promotion:** Collector-specific movement/classification of surviving objects into longer-lived storage. Promotion pressure can indicate survivor behavior or live-set growth, not automatically a leak. See Master Chapters 9 and 41.

Q

- **Queue:** Collection modeling a head chosen for inspection/removal under a policy. `offer/poll/peek` use special results, while `add/remove/element` can throw. See Master Chapter 29.

R

- **Race condition:** Correctness depends on uncontrolled relative timing. A race can exist without meeting the JMM's narrower definition of data race. See Master Chapter 38.
- **Record:** Final data-carrier class with declared components and generated accessors, constructor, equality, hash, and string form. Component references are only shallowly immutable. See Master Chapters 19 and 24.
- **Recursion:** Method/problem definition invoking itself on smaller subproblems with a base case. Stack depth and repeated work must be analyzed. See Master Chapters 8 and 47.
- **Reference type:** Class, interface, array, or type-variable category whose values are references or null. Representation of references is not fixed by the language. See Master Chapter 12.
- **Reflection:** Runtime inspection/invocation through metadata APIs. It can cross design boundaries, add access/configuration complexity, and should not be assumed to have ordinary call performance. See Master Chapter 21.
- **Region:** Collector-specific heap subdivision used by region-based collectors such as G1. Region size, roles, and humongous thresholds are implementation/configuration details. See Master Chapter 9.
- **Reification:** Runtime preservation of type information. Arrays are reified enough for store checks; most generic arguments are erased. See Master Chapter 22.
- **Retained size:** Heap-analysis estimate of memory that becomes unreachable if an object is removed, based on domination. It differs from direct shallow size. See Master Chapter 41.
- **Retry budget:** Explicit bound on attempts and elapsed time, ideally within one propagated deadline. It prevents retries from multiplying an outage. See Master Chapter 52.
- **Runtime constant pool:** Per-class/interface JVM runtime representation derived from the class-file constant pool, containing literals and symbolic references used in linking/execution. See Master Chapters 4 and 6.

S

- **Safepoint:** HotSpot mechanism/state permitting selected global VM operations when relevant threads are at safe locations. Reasons, polling, and pause behavior are implementation-specific. See Master Chapter 10.
- **Sealed class/interface:** Type restricting permitted direct subclasses/implementations, enabling controlled hierarchies and exhaustive reasoning. Permitted types must satisfy language/module/package rules. See Master Chapters 18 and 24.
- **Serialization:** Encoding data/object state for storage or transfer. Wire schema, compatibility, limits, validation, and trust matter more than convenient object mapping. See Master Chapters 32 and 50.
- **Shallow copy:** New outer object/collection whose referenced elements are shared. It protects outer structure but not mutable nested objects. See Master Chapter 19.
- **Shallow size:** Memory directly occupied by one heap object, excluding referenced objects. Large retained graphs can have a small shallow owner. See Master Chapter 41.
- **Short-circuiting:** Operation that can complete without consuming all input, such as `findFirst`, `anyMatch`, or Boolean `&&`. State/order can constrain savings. See Master Chapters 13 and 31.
- **SOLID:** Five object-design heuristics: single responsibility, open/closed, Liskov substitution, interface segregation, and dependency inversion. They guide trade-offs, not class-count targets. See Master Chapter 49.
- **Stack frame:** Per-invocation JVM frame containing local variables, operand stack, and linkage information conceptually. Frame layout is implementation-specific. See Master Chapters 6 and 8.
- **Starvation:** A task remains unable to acquire service/resource while others progress. Fairness, partitioning, priority aging, or capacity policy may mitigate it. See Master Chapter 38.
- **Static binding:** Compile-time selection not based on receiver overriding, as with static method hiding and overload resolution. Contrast with dynamic instance-method dispatch. See Master Chapters 14 and 17.
- **Stream:** Single-use lazy traversal pipeline, not a data container. Intermediate operations describe transformations; a terminal operation drives consumption. See Master Chapter 31.
- **Synchronization:** Coordination that protects invariants and establishes memory ordering. It includes locks, volatile, atomics, thread lifecycle edges, and higher-level concurrent structures. See Part V.
- **Synchronized:** Java keyword for intrinsic monitor acquisition/release around a block or method. Successful unlock happens-before a later successful lock of the same monitor. See Master Chapter 34.
- **System class loader:** Application class loader returned by `ClassLoader.getSystemClassLoader` in ordinary launches. Its exact implementation and hierarchy can vary. See Master Chapter 5.

T

- **Tail latency:** High-percentile response time, such as p99, reflecting slow requests hidden by averages. Always state traffic, window, errors, and sampling method. See Master Chapter 39.
- **Thread:** Java execution abstraction represented by `Thread`, with lifecycle, interruption, and memory-ordering rules. It may be a platform or virtual thread in Java 21. See Master Chapter 33.
- **Thread confinement:** Keeping mutable state accessible to only one thread at a time, eliminating shared-memory races for that state. Transfer still needs safe publication. See Master Chapter 38.
- **Thread local:** Per-thread value associated with a `ThreadLocal` key. Pooled-thread values require cleanup; massive virtual-thread use requires footprint awareness. See Master Chapters 33 and 41.
- **Thread pool:** Reuses a bounded/scaled set of worker threads to execute tasks, usually with a queue and rejection policy. Pool sizing follows workload and downstream capacity. See Master Chapter 36.
- **Thread safety:** Property that a component's documented invariants hold under allowed concurrent use. It must specify atomicity, traversal, visibility, and compound-operation boundaries. See Master Chapter 38.
- **Throughput:** Completed useful operations per unit time under stated load and correctness. Higher throughput can coexist with worse latency or resource cost. See Master Chapter 39.
- **Transaction:** Group of database operations committed or rolled back under a database's atomicity/isolation/durability contract. It does not automatically include remote services. See Master Chapter 50.
- **Transitive dependency:** Library pulled into a build through another dependency. Its version, license, vulnerabilities, and runtime presence remain application supply-chain concerns. See Master Chapter 51.
- **Treeification:** OpenJDK `HashMap` optimization that can convert a sufficiently collided bucket into a tree under thresholds. It is version-sensitive, not a `Map` contract. See Master Chapter 27.
- **Try-with-resources:** Statement managing `AutoCloseable` resources in reverse declaration order. Work failure remains primary and close failures become suppressed. See Master Chapter 20.
- **Type erasure:** Translation model removing most generic type arguments from runtime representation while adding casts/bridges as needed. It explains raw types and heap pollution. See Master Chapter 22.
- **Type inference:** Compiler derivation of omitted generic/lambda/local types from constraints and context. Inferred type follows language rules, not runtime value changes. See Master Chapters 22 and 24.

U

- **Unboxing:** Conversion from wrapper reference to primitive. Unboxing null throws `NullPointerException`; numeric overload and equality behavior can change across boxing boundaries. See Master Chapter 12.
- **Unicode code point:** Integer identifying a Unicode character value. Java `String` indexes UTF-16 code units, so one supplementary code point occupies two `char` values. See Master Chapter 15.
- **Unmodifiable view:** Wrapper rejecting mutation through that reference while often reflecting mutations through a backing alias. It is not necessarily an immutable snapshot. See Master Chapter 25.

V

- **Value-based class:** API-design designation for classes whose instances should be treated by value and whose identity should not be used for synchronization or identity-sensitive operations. See Master Chapter 19.
- **Variance:** Relationship between parameterized types when type arguments are related. Java uses invariant generic classes with `extends`/`super` wildcards for use-site covariance/contravariance. See Master Chapter 22.
- **Virtual thread:** Lightweight `Thread` scheduled by the Java runtime, suited to high-concurrency blocking workloads. It improves scale, not CPU speed, and does not remove downstream limits. See Master Chapter 37.
- **Volatile:** Field modifier providing defined visibility/order and atomic reads/writes for the field's value. Compound actions like increment remain non-atomic. See Master Chapters 11 and 35.

W

- **Wait set:** Conceptual monitor-associated set of threads that called `wait`. Notification permits competition to reacquire the monitor; condition loops must handle wakeups and recheck predicates. See Master Chapter 34.
- **Weak reference:** Reference that does not keep its referent strongly reachable and is cleared under GC reachability rules. It is not a deterministic cache eviction policy. See Master Chapters 9 and 41.
- **Work stealing:** Scheduling approach where idle workers take tasks from other workers' deques, used by fork/join execution. Blocking and task granularity affect efficiency. See Master Chapter 36.
- **Write skew:** Transaction anomaly where concurrent transactions read overlapping state and write disjoint rows, jointly violating an invariant under some snapshot isolation schemes. See Master Chapter 50.

Z

- **ZGC:** HotSpot low-latency concurrent collector. Generational mode was introduced in JDK 21; selection, flags, defaults, and behavior are release/vendor-specific. See Master Chapters 9 and 41.

SDE-2 CORE CHAPTER

Chapter 6 - Primary References and Further Reading

This appendix is a map to primary sources, not a list of links to memorize. Use it when a statement in this book needs a precise boundary: whether code is legal Java, what bytecode must mean, what an API promises, what a particular JDK implements, or how a build and test tool behaves. Unless a link explicitly says otherwise, the Java links below target Java SE or JDK 21, the baseline used by this book.

G.1 A Source Hierarchy for Interview-Quality Answers

Java documentation is published in several forms with different authority. Treating all of them as interchangeable leads to confident but inaccurate answers.

1. **Normative specifications** define the language and virtual machine. The Java Language Specification (JLS) answers questions such as whether an expression compiles, which overload is selected, and what synchronization orders memory actions. The Java Virtual Machine Specification (JVMS) defines class-file structure, loading, linking, initialization, instructions, and runtime data areas.
2. **Platform API specifications** define public contracts for Java SE types and methods. They are the first source for collection behavior, exception conditions, thread guarantees, stream requirements, and other library semantics. Read the entire class and method contract, including implementation requirements and API notes, while remembering that an implementation note is not necessarily a portable guarantee.
3. **OpenJDK JEPs** record the motivation, goals, alternatives, and delivery plan for JDK changes. A JEP is excellent design context, but it is not the final language or API specification. For a shipped feature, confirm the resulting contract in the JLS, JVMS, or Java SE API.
4. **JDK implementation and vendor documentation** describes tools and behavior of a distribution, commonly Oracle JDK or OpenJDK HotSpot. It is authoritative for that documented tool or implementation, not automatically for every Java implementation.
5. **Project documentation** is authoritative for Maven, Gradle, JUnit, JMH, and similar projects. Such documentation evolves independently of JDK 21. Pin the tool version used by a real repository and consult that version's manual.
6. **Source code** can explain an implementation, but it is not a substitute for a public contract. An interview answer should label implementation observations as such and avoid relying on internals unless the question explicitly asks about them.

A defensible answer often takes this form: "The API guarantees X; OpenJDK 21 commonly implements it using Y; code should rely on X." This separates portable reasoning from useful implementation knowledge.

G.2 Java SE 21 Specification Entry Points

- **Java Language Specification, Java SE 21** - Normative language specification. Use it for syntax, types, conversions, declarations, generics, overload resolution, expressions, exceptions, definite assignment, records, patterns, and the Java Memory Model.
- **Java Virtual Machine Specification, Java SE 21** - Normative JVM specification. Use it for class files, runtime data areas, loading, linking, initialization, verification, bytecode instructions, and execution semantics.
- **Java SE 21 specifications hub** - Official index for Java SE specifications plus JDK-specific specifications and tool manuals. Check the classification shown on the page rather than assuming every linked document is part of the language specification.
- **Java SE 21 API specification** - Public platform API contracts. Start here whenever the question is about an interface, class, method, exception, ordering guarantee, null policy, thread-safety statement, or performance characteristic promised by an API.
- **New API list in Java SE 21** - Version-specific inventory of added APIs. It is useful when separating Java 21 capabilities from APIs introduced earlier or later.
- **Deprecated API list in Java SE 21** - Official deprecation inventory and replacement guidance. Deprecation does not by itself mean immediate removal.
- **JDK 21 tool specifications** - Oracle JDK tool manuals for commands such as `java`, `javac`, `jcmd`, and `jfr`. These describe the JDK distribution and its tools; they are not the JLS.
- **`javac` command specification** - Compiler options, source compatibility, class paths, module paths, annotation processing, and `--release`. Use it to make build claims precise.
- **Java Object Serialization Specification for Java SE 21** - Serialization stream format, object graph rules, versioning, hooks, and security considerations. Prefer explicit formats for many new systems, but consult this specification when legacy Java serialization is actually in scope.

G.3 Language Topics in the JLS

The JLS is dense. The following direct chapter links make it practical during study and review.

- **Master Chapter 4: Types, Values, and Variables** - Primitive and reference types, type variables, parameterized types, reifiable types, intersections, arrays, and the relationship between compile-time types and runtime classes.
- **Master Chapter 5: Conversions and Contexts** - Widening, narrowing, boxing, unboxing, capture conversion, assignment conversion, invocation contexts, casting, and numeric promotion. Use it to resolve "does this compile?" questions rather than relying on intuition.
- **Master Chapter 6: Names** - Scope, shadowing, hiding, name classification, accessibility, and qualified names.
- **Master Chapter 8: Classes** - Fields, methods, constructors, inheritance, overriding, initialization order, nested classes, enum classes, and record classes.
- **Master Chapter 9: Interfaces** - Interface inheritance, default methods, functional interfaces, annotations, and annotation interfaces.
- **Master Chapter 10: Arrays** - Array covariance, runtime component types, creation, initialization, and access.
- **Master Chapter 11: Exceptions** - Checked and unchecked exceptions, exception analysis, abrupt completion, and precise rethrow.
- **Master Chapter 12: Execution** - Loading, linking, initialization, object creation, finalization terminology, and program exit. Pair it with JVMS Master Chapter 5 for runtime details.
- **Master Chapter 14: Blocks, Statements, and Patterns** - Control flow, switch statements and expressions, pattern variables, reachability, and definite assignment interactions.
- **Master Chapter 15: Expressions** - Evaluation order, method invocation, overload resolution, lambdas, method references, operators, casts, switch expressions, and pattern-related expressions.
- **Master Chapter 17: Threads and Locks** - Synchronization actions, happens-before, volatile variables, final-field semantics, wait sets, and the formal Java Memory Model. This is the primary source for visibility and ordering claims.
- **Master Chapter 18: Type Inference** - Constraint formulas and inference variables behind generic method calls, lambdas, method references, and diamond inference. Use it after first explaining the practical result in plain language.

For interview preparation, read the introductory portions and the exact subsection relevant to an example. Do not quote a nearby rule without checking its exceptions and defined terms.

G.4 JVM Topics in the JVMS

- **Master Chapter 2: JVM Structure** - Runtime data areas, frames, stacks, heaps, method areas, run-time constant pools, native stacks, and the JVM's abstract architecture. It deliberately leaves many implementation choices open.
- **Master Chapter 3: Compiling for the JVM** - Illustrative mappings from Java constructs to bytecode. It helps connect source-level reasoning to operand-stack execution, but does not mandate one compiler strategy for every construct.
- **Master Chapter 4: Class File Format** - Class-file layout, constant pools, descriptors, attributes, stack-map frames, verification constraints, and binary representation.
- **Master Chapter 5: Loading, Linking, and Initializing** - Class and interface loading, verification, preparation, resolution, class-loader constraints, and initialization. Use it for class-loader identity and initialization-trigger questions.
- **Master Chapter 6: JVM Instruction Set** - Normative definitions of bytecode instructions, their operands, stack effects, and exceptional behavior.
- **Master Chapter 7: Opcode Mnemonics** - Compact opcode reference useful while reading `javap` output.
- **Oracle `javap` manual for JDK 21** - Tool documentation for disassembling class files. Use `javap -c -v` as an observation aid, then interpret output using the JVMS.

The JVMS describes an abstract machine, not HotSpot's exact memory layout or JIT algorithms. Statements about TLABs, compressed object pointers, tiered compilation, or a collector's regions belong to implementation documentation and should be labeled accordingly.

G.5 Java 21 Feature Design Records

These OpenJDK JEP pages are primary design records. They explain goals and tradeoffs well, but the final JLS and API remain the contract for shipped behavior.

- **JEP 431: Sequenced Collections** - Motivation and design for uniform first/last/reversed operations across ordered collections. Follow with the Java SE 21 `SequencedCollection`, `SequencedSet`, and `SequencedMap` API contracts.
- **JEP 439: Generational ZGC** - Rationale and goals for adding generations to ZGC in JDK 21. Use the JDK's GC documentation and measurements from the target workload before recommending a collector.
- **JEP 440: Record Patterns** - Final design for record-pattern deconstruction in Java 21. Confirm typing, applicability, and match behavior in the JLS.
- **JEP 441: Pattern Matching for `switch`** - Final design for type patterns, guarded cases, null handling, dominance, and enhanced exhaustiveness in Java 21. Use JLS Master Chapters 14 and 15 for normative rules.
- **JEP 444: Virtual Threads** - Goals, scheduling model, observability, and migration guidance for virtual threads finalized in JDK 21. Pair it with the `Thread` API and Oracle's virtual-thread guide.

Earlier feature records often referenced by SDE-2 interviews include **JEP 361: Switch Expressions**, **JEP 394: Pattern Matching for `instanceof`**, **JEP 395: Records**, and **JEP 409: Sealed Classes**. Their historical examples clarify design intent; use the Java 21 JLS for the consolidated rules.

G.6 Core Library API Maps

- **`java.util` package specification** - Collections, comparators, optionals, random generation, utilities, and common value types. Read the collection-interface contracts before relying on a concrete implementation.
- **`SequencedCollection` API**, **`SequencedSet` API**, and **`SequencedMap` API** - Java 21 contracts for encounter-ordered collection operations and reversed views.
- **`java.util.stream` package specification** - Stream pipelines, laziness, non-interference, statelessness, ordering, reduction, collectors, and parallel execution. It is the primary source for behavioral constraints on stream lambdas.
- **`java.util.concurrent` package specification** - Executors, queues, synchronizers, concurrent collections, atomics, futures, and package-level memory-consistency guarantees.
- **`Thread` API for Java 21** - Platform and virtual thread creation, lifecycle, interruption, joining, uncaught exceptions, and method contracts.
- **`Executors` API** - Executor factory contracts, including the virtual-thread-per-task executor. Factory convenience does not remove the need to reason about admission control and downstream capacity.
- **`CompletableFuture` API** - Completion-stage composition, execution policies, cancellation behavior, and exceptional completion.
- **`java.nio` package specification** - Buffers, channels, charsets, non-blocking I/O foundations, and related contracts.
- **`Files` API** - File operations, streams over directories and lines, copy/move behavior, attributes, and resource-closing requirements.
- **`ByteBuffer` API** - Position, limit, capacity, slicing, byte order, heap versus direct buffers, and view semantics.
- **`java.io` package specification** - Byte and character streams, readers, writers, serialization APIs, and closeable resources.
- **`java.time` package specification** - Immutable date-time types, time zones, clocks, durations, periods, parsing, and thread-safety guidance.
- **`java.sql` package specification** and **`javax.sql` package specification** - JDBC contracts, data sources, connections, statements, result sets, transactions, and row sets. Database isolation and locking behavior also require the target database's documentation.

When an API page states average or expected complexity, null handling, optional operations, encounter order, or concurrency guarantees, cite that statement. Otherwise, present complexity as an implementation-specific expectation and name the implementation.

G.7 Virtual Threads and Concurrency Guidance

- **Oracle Java 21 virtual threads guide** - JDK 21 guidance on creating virtual threads, representing tasks, scheduling, pinning, thread-local usage, and observability. This is Oracle JDK guidance, while JEP 444 and the API describe design and contract.
- **Java Memory Model in JLS 17.4** - Normative model for inter-thread actions, synchronization order, happens-before, data races, and correctly synchronized programs.
- **`Lock` API** - Explicit-lock semantics and memory synchronization effects. Read each implementation's fairness, interruptibility, and condition behavior separately.
- **`AtomicInteger` API** and **`VarHandle` API** - Atomic operations and explicit memory-ordering modes. Prefer the simplest ordering that is demonstrably correct; do not infer compound-operation atomicity from individual atomic calls.

For concurrency questions, give the invariant first, identify the synchronization edge that protects it, and then cite the applicable API or JLS rule. A schedule that "usually works" is not a correctness proof.

G.8 HotSpot, Garbage Collection, and Runtime Tuning

- **Java 21 HotSpot Virtual Machine guide** - Oracle documentation for the HotSpot implementation shipped with JDK 21. Use it for runtime implementation topics, not as a universal JVM specification.
- **Java 21 HotSpot Garbage Collection Tuning Guide** - Collector selection, ergonomics, generations, pause goals, throughput, footprint, and tuning workflow for HotSpot. Treat flags and defaults as JDK-version-specific.
- **G1 Garbage Collector tuning** - Oracle guidance for diagnosing and tuning G1. Begin with evidence from GC logs and workload goals before changing flags.
- **JEP 439: Generational ZGC** - JDK 21 design context for generational ZGC. A JEP's goals are not a latency guarantee for a specific service.
- **OpenJDK 21 GA source tree** - Primary implementation source for OpenJDK 21. It can answer implementation questions after the specification boundary is clear. Internal classes and algorithms may change without preserving source-level compatibility.

A tuning claim should name the JDK build, collector, heap constraints, allocation rate, traffic profile, and measured outcome. Avoid universal prescriptions such as "collector X is always faster" or "larger heaps always reduce latency."

G.9 Diagnostics, JFR, JMC, and jcmd

- **Java 21 troubleshooting diagnostic tools** - Oracle's overview of command-line and graphical diagnostic facilities. It is a good starting map during an incident.
- **`jcmd` manual for JDK 21** - Authoritative Oracle JDK command documentation for sending diagnostic requests to a running JVM. Obtain the commands supported by the target process rather than assuming every build exposes the same set.
- **`jfr` command manual for JDK 21** - Recording-file inspection, summary, printing, assembly, disassembly, and metadata commands.
- **JDK Flight Recorder API Programmer's Guide** - Oracle guide to consuming recordings and creating custom JFR events using the `jdk.jfr` API.
- **JFR recording configurations** - Guidance on predefined and custom recording settings. Choose settings based on diagnostic goals and validate overhead in the target environment.
- **JDK Mission Control documentation** - Oracle documentation and releases for JMC, a separately distributed tool for inspecting JFR data and JVM behavior. It is not part of the Java language specification and may have its own compatibility matrix.
- **`jstack` manual for JDK 21** and **`jmap` manual for JDK 21** - JDK tool references for thread and memory diagnostics. Their manuals label these tools experimental and unsupported; prefer supported diagnostic paths such as appropriate `jcmd` operations where available.

During diagnosis, capture evidence before tuning: timestamps, service symptoms, JDK/build, command used, recording duration, load conditions, and relevant configuration. Correlate JFR, GC, application, and infrastructure signals instead of drawing a conclusion from one isolated sample.

G.10 Microbenchmarking with JMH

- **OpenJDK JMH project** - Official project page for the Java Microbenchmark Harness. JMH is an OpenJDK Code Tools project, not a Java SE API.
- **Official JMH repository and README** - Build instructions, supported modes, annotations, and project guidance. Use the generated harness and the recommended build setup.
- **Official JMH samples** - Executable examples covering dead-code elimination, constant folding, state scope, setup, parameters, profilers, and other benchmark traps.

JMH handles much of the measurement machinery, but it cannot decide whether a benchmark represents a production workload. State the hypothesis, isolate the operation carefully, consume results, parameterize realistic inputs, inspect allocation and generated behavior where relevant, use forks, report uncertainty, and retain the full environment. Benchmark results are evidence for the tested configuration, not timeless truths about a Java construct.

G.11 Security and Serialization

- **Java SE 21 security developer guide** - Oracle's versioned guide to Java security APIs and mechanisms, including cryptography, providers, secure communication, authentication, and related topics.
- **Java security overview for JDK 21** - Architecture and terminology for security providers, cryptographic services, public-key infrastructure, secure communication, and access control.
- **Oracle Secure Coding Guidelines for Java SE** - Vendor-maintained secure coding guidance. It is useful for threat-aware design but is updated independently of the JLS and should be checked for its applicable platform scope.
- **Java serialization filters guide for JDK 21** - Oracle guidance for allowlists, reject lists, resource limits, and filter factories when native serialization cannot be avoided.
- **Java Object Serialization Specification** - The detailed stream and object-graph contract. Use it alongside, not instead of, a threat model for untrusted input.

Security behavior changes with JDK updates, library versions, deployment configuration, and threat intelligence. For a real system, also consult supported-version advisories and the primary documentation for the framework, container, operating system, identity provider, and protocol in use.

G.12 Maven Primary Documentation

- **Maven build lifecycle guide** - Default, clean, and site lifecycles; phases; goals; packaging; and safe command selection.
- **Maven dependency mechanism guide** - Transitive dependencies, mediation, scopes, management, optional dependencies, and exclusions.
- **Maven reproducible builds guide** - Project configuration and verification practices for reproducible artifacts.
- **Maven Wrapper documentation** - Running a project with a declared Maven distribution rather than depending on an arbitrary machine-wide installation.

Maven's online documentation can describe the current Maven line rather than the version in an older repository. Confirm the wrapper or CI version, plugin versions, effective POM, active profiles, and repository settings before diagnosing resolution or lifecycle behavior.

G.13 Gradle Primary Documentation

- **Gradle User Manual** - Official entry point for builds, tasks, plugins, dependency management, performance, authoring, and reference material.
- **Gradle dependency management** - Configurations, variants, metadata, repositories, constraints, platforms, and dependency resolution.
- **Gradle dependency locking** - Lock-state generation, update, use, and limitations for repeatable resolution.
- **Gradle dependency verification** - Checksums, signatures, trust configuration, and verification metadata for supply-chain protection.
- **Gradle Java toolchains** - Declaring, discovering, selecting, and provisioning Java installations for compilation, testing, and execution.
- **Gradle security guidance** - Official guidance on wrapper integrity, dependency verification, credentials, and other build-security concerns.

The `current` URLs follow Gradle's current release. For a repository, use its wrapper version and switch the documentation selector to that exact version before relying on DSL behavior or defaults.

G.14 JUnit Primary Documentation

- **JUnit official site** - Project entry point for releases, documentation, community information, and related modules.
- **JUnit User Guide** - Official guide for the JUnit Platform, Jupiter programming and extension models, parameterized tests, suites, build integration, IDE support, parallel execution, and migration topics.

The `current` guide moves with JUnit releases. Pin the version used by the build and open the matching versioned guide for exact annotations, engines, launcher APIs, and configuration parameters. In an interview, distinguish the JUnit Platform, a test engine such as Jupiter, and the assertions or mocking libraries selected by the project.

G.15 Suggested Reading Routes

Use a route based on the question rather than reading every source front to back.

- **Language puzzle:** Compile a minimal example with JDK 21, then consult JLS Master Chapters 4, 5, 8, 14, 15, or 18. Explain the rule and one practical implication.
- **Memory visibility or race:** State the shared invariant, identify synchronization actions, use JLS 17.4 and the relevant `java.util.concurrent` contract, and test only as supporting evidence.
- **Collection behavior:** Start at the interface contract, then the concrete API, then JEP 431 if design motivation matters. Do not infer a guarantee merely from current source code.
- **Class loading or bytecode:** Use JVMS Master Chapters 4 through 6. Use `javap` to inspect an example and label OpenJDK-specific observations.
- **Java 21 feature:** Read the relevant JEP for motivation, then the JLS or API for final semantics, and finally a JDK guide for operational advice.
- **GC or latency incident:** Read the HotSpot VM and GC guides, collect JFR and GC evidence with documented tools, and evaluate the exact deployed JDK under representative load.
- **Microbenchmark:** Start with JMH's README and samples. Pre-register the question, preserve benchmark code and environment, and report distributions or uncertainty rather than only the best score.
- **Build failure:** Identify the wrapper and runtime versions, then use the exact Maven or Gradle documentation. Capture dependency insight, effective configuration, repositories, and toolchain selection.
- **Test design:** Use the version-matched JUnit guide, separate unit from integration boundaries, and make assertions about externally meaningful behavior.
- **Security decision:** Start with a threat model and current supported-version guidance. Use the Java security and serialization sources for platform behavior, then consult the primary documentation of every external component involved.

The final habit is version discipline. A precise source from the wrong JDK, collector, database, build tool, or test framework can still produce the wrong answer. Record the relevant versions, distinguish guarantees from observations, and prefer the smallest authoritative source that directly answers the question.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: rapid recall and code-output questions
- Interview Core: timed coding and Java deep dives
- SDE-2 Follow-up: design and production incidents
- Challenge: run full mixed mock loops under realistic timing

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: JAVA 08 - Advanced Java E: Performance, Diagnostics, and GC Incidents

Series index

Return to the complete series index

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, or System Design and Backend first, then follow the books inside that segment in order. Stable study-step codes remain on filenames and links; the clearer segment book codes appear on the website and every PDF cover.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Advanced Java B: Language, OOP, and Modern Java
Java	**JAVA 05**	Advanced Java C: Collections, Streams, and I/O
Java	**JAVA 06**	Advanced Java A: JVM and Execution
Java	**JAVA 07**	Advanced Java D: Concurrency and the Memory Model
Java	**JAVA 08**	Advanced Java E: Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Advanced Java G: Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
System Design	**SD 01**	Advanced Java F: Design, Backend, Testing, and Security
System Design	**SD 02**	MySQL for Java Backend Interviews
System Design	**SD 03**	Hibernate and JPA for Java Backend Interviews
System Design	**SD 04**	Spring Framework for Java Engineers
System Design	**SD 05**	Spring Boot for Java Backend Engineers

SEGMENT	BOOK	FOCUSED LEARNING PATH
System Design	**SD 06**	Spring Data for Java Backend Engineers
System Design	**SD 07**	MongoDB for Java Backend Interviews
System Design	**SD 08**	Redis for Java Backend Interviews
System Design	**SD 09**	Apache Kafka and Spring Kafka
System Design	**SD 10**	Spring Ecosystem Extensions
System Design	**SD 11**	Spring AI for Java Engineers
System Design	**SD 12**	Advanced Java H: Spring Boot and REST APIs
System Design	**SD 13**	Advanced Java I: Persistence, SQL, JPA, and Caching
System Design	**SD 14**	Advanced Java J: Distributed Systems and System Design

Roadmap editions identify planned books whose chapter sets will be expanded one at a time. Existing deep-dive books remain available later in each segment, so no published material is displaced.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Java Engineering - Book 09 of 09 - Study Step 18G. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.